

Towards Efficient Generative Large Language Model Serving: A Survey from Algorithms to Systems

XUPENG MIAO, Purdue University, West Lafayette, United States

GABRIELE OLIARO, Carnegie Mellon University, Pittsburgh, United States

ZHIIHAO ZHANG, Carnegie Mellon University, Pittsburgh, United States

XINHAO CHENG, Carnegie Mellon University, Pittsburgh, United States

HONGYI JIN, Carnegie Mellon University, Pittsburgh, United States

TIANQI CHEN, Carnegie Mellon University, Pittsburgh, United States

ZHIIHAO JIA, Carnegie Mellon University, Pittsburgh, United States

In the rapidly evolving landscape of artificial intelligence (AI), generative large language models (LLMs) stand at the forefront, revolutionizing how we interact with our data. However, the computational intensity and memory consumption of deploying these models present substantial challenges in terms of serving efficiency, particularly in scenarios demanding low latency and high throughput. This survey addresses the imperative need for efficient LLM serving methodologies from a machine learning system (MLSys) research perspective, standing at the crux of advanced AI innovations and practical system optimizations. We provide in-depth analysis, covering a spectrum of solutions, ranging from cutting-edge algorithmic modifications to groundbreaking changes in system designs. The survey aims to provide a comprehensive understanding of the current state and future directions in efficient LLM serving, offering valuable insights for researchers and practitioners in overcoming the barriers of effective LLM deployment, thereby reshaping the future of AI.

CCS Concepts: • **Computing methodologies** → **Machine learning**; **Parallel computing methodologies**; **Natural language processing**; • **General and reference** → **Surveys and overviews**; • **Hardware** → **Analysis and design of emerging devices and systems**; **Emerging technologies**; • **Computer systems organization** → **Architectures**; • **Software and its engineering** → **Operating systems**;

Additional Key Words and Phrases: Large language model, efficiency, algorithm, system, inference, serving

ACM Reference Format:

Xupeng Miao, Gabriele Oliaro, Zhihao Zhang, Xinhao Cheng, Hongyi Jin, Tianqi Chen, and Zhihao Jia. 2025. Towards Efficient Generative Large Language Model Serving: A Survey from Algorithms to Systems. *ACM Comput. Surv.* 58, 1, Article 15 (September 2025), 37 pages. <https://doi.org/10.1145/3754448>

Authors' Contact Information: Xupeng Miao, Purdue University, West Lafayette, Indiana, United States; e-mail: xupeng@purdue.edu; Gabriele Oliaro, Carnegie Mellon University, Pittsburgh, Pennsylvania, United States; e-mail: goliaro@cs.cmu.edu; Zhihao Zhang, Carnegie Mellon University, Pittsburgh, Pennsylvania, United States; e-mail: zhihaoz3@cs.cmu.edu; Xinhao Cheng, Carnegie Mellon University, Pittsburgh, Pennsylvania, United States; e-mail: xinhaoc@andrew.cmu.edu; Hongyi Jin, Carnegie Mellon University, Pittsburgh, Pennsylvania, United States; e-mail: hongyij@cs.cmu.edu; Tianqi Chen, Carnegie Mellon University, Pittsburgh, Pennsylvania, United States; e-mail: tqchen@cmu.edu; Zhihao Jia, Carnegie Mellon University, Pittsburgh, Pennsylvania, United States; e-mail: zhihao@cmu.edu.



This work is licensed under a Creative Commons Attribution 4.0 International License.

© 2025 Copyright held by the owner/author(s).

ACM 0360-0300/2025/09-ART15

<https://doi.org/10.1145/3754448>

1 Introduction

Generative **large language models (LLMs)** have become a driving force behind significant advancements in **artificial intelligence (AI)** and have demonstrated exceptional performance across a wide range of language-related tasks. From machine translation to sentiment analysis, question answering, and text generation, these models have shown their prowess in understanding, generating, and manipulating human languages. The advent of Transformer-based architectures, such as **Generative Pre-trained Transformer-family (GPT)** [31], LLaMA-family [290], DeepSeek-family [121, 190], and other latest public LLMs (e.g., OPT [354], BLOOM [305], Mistral [148], DeciLM [75], Baichuan [326], and GLM [343]) has played a pivotal role in this paradigm shift, revolutionizing the way **natural language processing (NLP)** tasks are approached. Beyond NLP, these models are also transforming a wider range of applications, including automated programming [57], science discovery [156], personalized digital assistants [84], creative arts [248], and next-generation computing architecture [232], demonstrating their versatility and profound impact across various industries.

However, the unprecedented success of LLMs has also given rise to several challenges, most notably, their formidable computational requirements during serving. The immense model size and complexity, coupled with the need for extensive computational resources, have impeded their widespread deployment in real-world applications. The resource-intensive nature of these models raises concerns over energy consumption, scalability, and accessibility, hindering their adoption in broader communities without rich compute resources like large companies.

This survey paper aims to address the critical need for efficient LLM serving and presents an exhaustive exploration of the existing multifaceted strategies proposed by the research community to tackle this challenge. We present an in-depth examination of the entire spectrum of solutions, spanning from algorithmic innovations to novel system architectures, all aimed at optimizing the inference process for LLMs.

1.1 Objectives

The primary objective of this survey is to provide a comprehensive overview of the latest advancements in LLM serving and inference. We will systematically review and categorize the existing techniques based on their underlying approaches, highlighting their strengths and limitations. The survey will cover a broad range of methodologies, encompassing decoding algorithm, architecture design, model compression, low-bit quantization, parallel computation, memory management, request scheduling, and kernel optimization.

1.2 Structure

The article is structured as follows: Section 2 introduces the background information about LLM serving. Section 3 includes our taxonomy of existing approaches on efficient LLM serving and revisits these related works from two aspects: algorithmic innovations (Section 3.1) and system optimizations (Section 3.2). After that, we list some representative LLM serving frameworks and provide analysis in Section 4. Section 5 discusses benchmarks of LLM serving systems. Section 6 clarifies the connection between this survey and other related literature. Finally, we propose some promising exploration directions in Section 7 for improving generative LLM serving efficiency to motivate future research.

2 Background

2.1 Transformer-based LLM

Transformer-based LLMs have marked a significant shift in the field of NLP, introducing a new paradigm for understanding and generating human language. Central to this innovation is the Transformer architecture, which is built upon the concept of self-attention mechanisms [296],

allowing the model to weigh the importance of different parts of the input data when making predictions. Mathematically, the self-attention mechanism in Transformers can be described as follows: For an input sequence $X = [x_1, x_2, \dots, x_n]$, the Transformer computes a set of queries Q , keys K , and values V using linear transformations of X . The self-attention scores are then computed as:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V, \quad (1)$$

where d_k is the dimension of the keys. This mechanism allows the model to focus on different parts of the input sequence for each element of the output, capturing complex dependencies regardless of their distance in the input sequence.

Another important structure in Transformers is the **Feed-Forward Network (FFN)**, which is present in each layer of the Transformer and significantly contributes to its computational intensity. The FFN typically consists of two linear transformations with a non-linear activation function in between, usually represented as:

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2. \quad (2)$$

Here, W_1 , W_2 , b_1 , and b_2 are learnable parameters of the FFN, and the non-linear function $\max(0, \cdot)$ (ReLU, in this case) introduces the necessary non-linearity into the model, allowing it to learn more complex patterns. The FFN is responsible for a significant portion of the model's parameter count and, consequently, its memory footprint and computational load. In each Transformer layer, after the **multi-head attention (MHA)** aggregates information from different parts of the input, the FFN processes this aggregated information independently for each position. This parallel processing capability is a key strength of the Transformer, allowing it to handle sequences effectively. However, it also means that the computational load and memory requirements scale with the length of the input sequence and the depth of the network.

The combination of self-attention and FFN in Transformer-based LLMs enables these models to capture a wide range of linguistic contexts and nuances, setting new benchmarks in various NLP tasks. However, the substantial computational requirements for training and inference have become a critical area of research, focusing on optimizing these aspects without significantly compromising performance. The Transformer model also includes other key components like position encoding, which adds information about the position of each token in the sequence, and the MHA mechanism, which allows the model to focus on different parts of the sequence in different representational spaces.

2.2 GPUs and Other Accelerators

The rapid advancement of LLMs owes much to the evolution of GPU architecture and other accelerators, which are integral to enhancing model performance and efficiency. **Graphics Processing Units (GPUs)** have emerged as a cornerstone in this field, primarily due to their superior parallel processing capabilities. Unlike traditional CPUs, which are designed for sequential processing, GPUs consist of thousands of small, efficient cores designed for handling multiple tasks simultaneously. This makes them exceptionally well-suited for the matrix and vector operations that are ubiquitous in deep learning computations, especially for Transformer-based models.

A typical GPU architecture comprises an array of **Streaming Multiprocessors (SMs)**, each containing several cores that share a common instruction unit but can execute independent threads in parallel. Additionally, the shared memory (SRAM) within each SM allows for efficient data exchange and synchronization among threads, significantly optimizing the memory access patterns required in LLM computations. This design is particularly beneficial for the computationally intensive tasks in LLMs, such as the calculations of self-attention and FFNs in Transformers. GPUs

also come equipped with **high-bandwidth memory (HBM)**, which allows for faster data transfer rates, significantly reducing the bottleneck associated with memory access during large-scale computations. Moreover, the latest GPU architectures, such as NVIDIA's Ampere and Hopper architectures, continue to offer enhancements and push the boundaries of LLM computation, such as improved memory bandwidth and capacity, higher **floating-point operations per second (FLOPS)**, specialized mixed-precision computing units (i.e., Tensor Core) and more efficient utilization of resources, further accelerating the performance of LLMs. Some of them support various precision formats, including FP32 (32-bit floating point), TF32 (TensorFloat-32), FP16 (16-bit floating point), BF16 (Brain Floating Point), and even INT8/INT4, allowing for flexible trade-offs between computational speed and numerical precision, essential in optimizing LLM performance.

Beyond GPUs, a vast array of hardware platforms have been explored for LLM deployment, encompassing CPUs [19, 262], mobile and edge devices [79], ASICs [239, 367], as well as specialized accelerators such as TPUs [157], FPGAs [336], and other emerging AI chips from various manufacturers (e.g., Apple M2 Ultra [172], AWS Inferentia [6], SambaNova [28], Cerebras [81], and Graphcore IPU [16]). This survey primarily underscores research anchored in the use of GPUs, and several technical motivations drive this emphasis. Due to their architectural innovations and superior computational power, GPUs have dominated the research area of large-scale deep learning in the past few years [44]. Furthermore, the programming languages of GPUs, like NVIDIA's CUDA and AMD's ROCm, facilitate a fine-grained control over thread hierarchies, allowing researchers to exploit the massive parallelism inherent in GPUs. It attracts numerous developers to build mature software ecosystems on top of these GPUs, fostering a majority of the seminal and advanced LLM research. While other hardware platforms indeed bring unique strengths to specific contexts, the vast reservoir of research, development, and deployment centered around GPUs makes them an indispensable reference for an in-depth comprehension of LLM inference methodologies. Considering the hardware similarities, other hardware platforms can also benefit from the design philosophies, insights, and methodologies discussed in this survey.

2.3 LLM Inference

LLM inference, particularly in models like GPT, often employs an auto-regressive decoding approach. This method is central to how these models generate text, ensuring that each new word or token produced takes into account the entire sequence generated so far. Auto-regressive decoding operates under the principle of sequentially predicting the next token in a sequence, given all the previous ones, as shown in Algorithm 1.

Here, $P(y|X_{t-1})$ represents the probability of the next token y given the current sequence X_{t-1} , and $\oplus$ denotes the concatenation operation. The argmax function is used to select the most probable next token at each step.

This auto-regressive approach is fundamental in LLM inference for generating coherent and contextually appropriate text. It ensures that each token generated is conditioned on a comprehensive understanding of all previously generated content, allowing LLMs to produce highly relevant and fluent text sequences. Prior studies have provided in-depth analysis on the algorithmic intensity of Transformer-based LLM inference (e.g., counting the FLOPS, I/O and memory consumption) and extensive empirical results on cost estimation (e.g., modeling the inference latency [53]) according to the auto-regressive decoding algorithm execution. The optimization of LLM inference is a complex problem as there can be different optimal strategies with different algorithm configurations and system setups.

2.4 Challenges

This section describes a variety of challenges for efficient LLM serving.

ALGORITHM 1: Auto-Regressive Decoding for LLM Inference

```

1: Initialize the input sequence  $X_0$  with a given context or start token
2: for  $t = 1$  to  $T$  do
3:   Predict the next token  $y_t = \operatorname{argmax}_y P(y|X_{t-1})$ 
4:   Update the input sequence  $X_t = X_{t-1} \oplus y_t$ 
5:   if  $y_t$  is EOS then
6:     break

```

- **Latency and Response Time** Efficient LLM inference requires achieving low-latency and fast response times, especially in real-time applications like chatbots, virtual assistants, and interactive systems. Balancing model complexity with inference speed is a critical challenge that necessitates optimizing algorithms and system architectures to minimize response time without compromising accuracy.
- **Memory Footprint and Model Size** LLMs come with significant memory requirements due to their size and the vast number of parameters they contain. Deploying such models on memory-constrained devices poses a challenge, demanding the development of effective model compression techniques and system optimizations to reduce memory footprint without sacrificing performance.
- **Scalability and Throughput** Inference systems often face varying levels of request loads in production environments. Ensuring scalability and high throughput to handle multiple simultaneous requests efficiently requires parallel computation, request scheduling, and other system-level optimizations to distribute computational workload effectively across resources.
- **Hardware Compatibility and Acceleration** Efficiently leveraging hardware resources is crucial for LLM inference. Adapting LLM models to diverse hardware platforms and architectures, including CPUs, GPUs, and specialized accelerators, demands hardware-aware algorithm design and optimization to exploit the full potential of the underlying hardware.
- **Trade-offs between Accuracy and Efficiency** Optimizing the efficiency of LLM inference may sometimes involve trade-offs with model accuracy. Striking the right balance between model size, computational complexity, and performance is a challenging task that requires careful consideration and evaluation of various algorithmic and system-level techniques.

3 Taxonomy

Figure 1 illustrates our taxonomy of existing efforts on improving the LLM serving efficiency, which can be broadly classified into two categories, including algorithmic innovations and system optimizations. We will discuss each category in details individually.

3.1 Algorithmic Innovation

This section presents a comprehensive analysis of the various algorithms and techniques proposed to optimize language model inference efficiency. These works are proposed to address the native performance flaws of large-scale Transformer models through algorithmic advancements.

3.1.1 Decoding Algorithm. In this section, we review novel decoding algorithms as shown in Figure 2 that optimize the inference process of LLMs. These algorithms seek to reduce computational complexity and enhance the overall efficiency of language model inference during generation tasks.

- **Non-autoregressive decoding.** A major limitation of existing LLMs is the default autoregressive decoding mechanism, which *sequentially* generates output tokens one by one. To address this issue, one representative line of work is to abandon the autoregressive

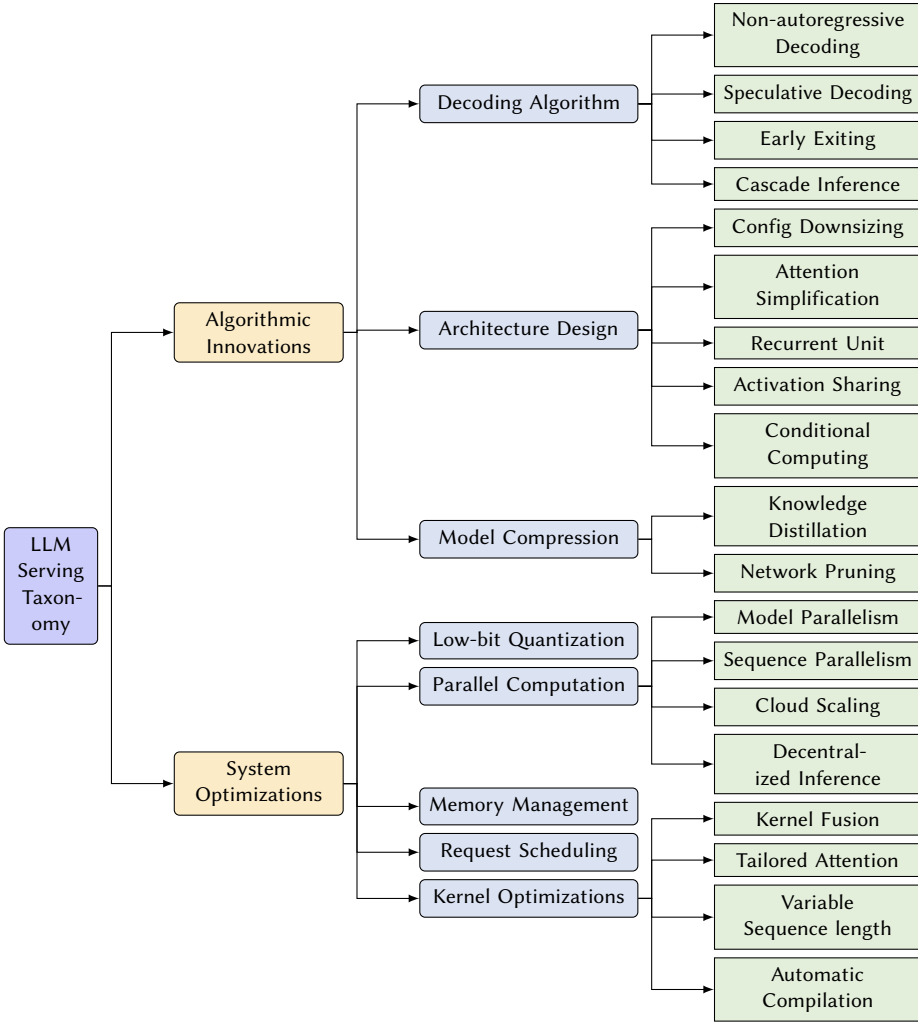


Fig. 1. Taxonomy of LLM inference advancements.

generation paradigm and decode the output tokens *in parallel*. Non-autoregressive decoding [110, 117, 122] is first proposed for machine translation acceleration by breaking the word dependencies during decoding and assuming a certain degree of conditional independence. To alleviate the translation quality reduction, some follow-up studies like semi-autoregressive decoding [111], further extend these non-autoregressive methods to reach auto-regressive model quality by modeling output dependencies [118, 347] or iteratively refining output tokens [173]. Blockwise parallel decoding [272] inserts a single feedforward layer to the base LLM to make predictions for multiple future positions in parallel, then backs off to the longest prefix validated by the base model. However, these approaches require to costly reproduce a new LLM with the new dependencies or tune partial layers of the original LLM, which are not always possible. Some recent efforts have been dedicated to generate multiple tokens at one decoding step without any training or modification to the model. Parallel decoding [257] reframes the greedy auto-regressive decoding as a system of nonlinear equations

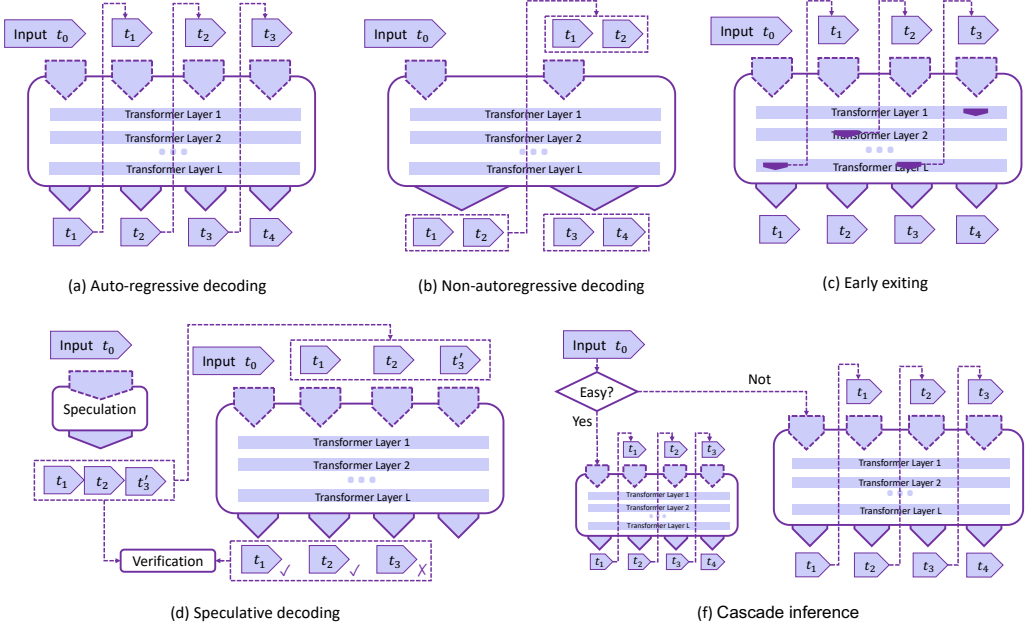


Fig. 2. Illustration of different LLM decoding algorithms

solvable in parallel leveraging Jacobi and Gauss-Seidel fixed-point iteration methods for fast inference. A thorough survey on non-autoregressive translation [319] has been proposed to summarize the recent advances in this direction. Until now, due to the unawareness of the conditional dependence between output tokens, the output quality of most of non-autoregressive methods has been still less reliable than the auto-regressive method despite an improvement in decoding speed.

- **Speculative decoding.** Another line of work addresses the sequential execution limitation by leveraging *speculative execution* [50] and improving decoding parallelism. Each decoding step during the autoregressive LLM inference process can be treated as the execution of a program with conditional branches, such as deciding which token to generate next. Speculative decoding [54, 177] has been proposed to make decoding predictions of multiple steps first in an efficient manner (e.g., using a smaller draft model with fewer model parameters) and verify these predictions simultaneously with the LLM. However, there are still several practical challenges remaining when applying speculative decoding to LLMs, e.g., how to make decoding predictions light-weight and accurate enough and how to achieve efficient parallel verification using LLMs. As shown in Figure 3, SpecInfer [211] first addresses these challenges by introducing a novel tree-based speculative inference and token verification mechanism and proposes a low-latency LLM serving system implementation (Section 4). The main advantage of speculative decoding is that it increases the parallelism without any changes to the outputs. Such guarantee comes from the fact that the predicted output is always verified by the original LLM and the fallback mechanism [165] takes effect when prediction goes wrong. The tree-based speculative decoding design has been directly adopted by numerous subsequent works, such as Medusa [51], EAGLE [183], and so on [38, 132, 197, 219, 229, 271, 280, 323, 358, 370]. Some recent efforts [61, 137, 185, 297] further explore how to adaptively generate better draft token tree structures to improve the speculation efficiency.

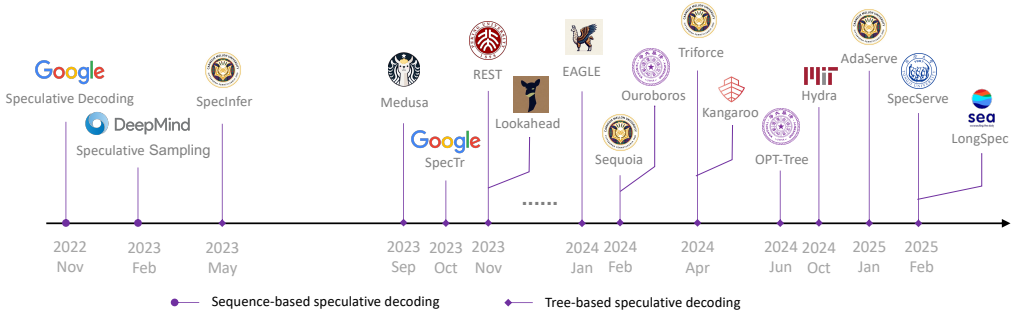


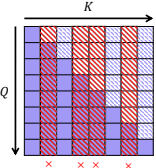
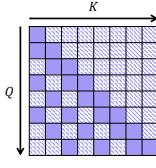
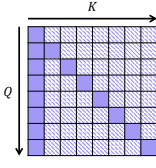
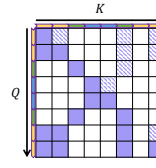
Fig. 3. Illustration of different speculative decoding approaches.

- **Early exiting.** Some other studies attempt to utilize the deep multi-layer architecture of existing LLMs and leverage the *early exiting* [286] mechanism to accelerate the decoding process. The intuition is that the output of early model layers has the potential to infer the target distribution confidently. They can emit predictions based on internal classifiers instead of running the whole LLM, and various exit conditions have been explored [131, 167, 187, 196, 278, 321, 333, 344, 368]. They are also called by *adaptive computation* [76, 259] since they adjust the amount of computation per request to amortize the total inference cost, i.e., taking less computation for easier inference requests. Kangaroo [191] applies this early exiting strategy to utilize LLM’s sub-network and construct a self-drafting model for speculative decoding. Broadly, these approaches are mostly restricted to the insufficient information carried by internal representations and may not faithfully making accurate predictions.
- **Cascade inference.** Driven by the varying complexities of inference requests, cascade inference employs a suite of LLMs of differing scales to minimize response time. Instead of directly using a massive model for every query, CascadeBERT [179] involves a series of internal classifiers corresponding to different model depths, organizes them in a cascading manner and adaptively selects proper ones based on the instance difficulty. Tabi [301] optimizes for serving discriminative models (i.e., not generative LLMs), but it takes a similar approach to incorporate small models and LLMs to handle queries with different confidence. FrugalGPT [56] leverages a learning-based approach to adaptively assign queries to different LLM APIs, optimizing both cost and performance. A concurrent work [372] jointly optimizes model multiplexing and query caching and also analyzes the optimality of minimizing inference cost. Mixture-of-thought [341] extends the cascade idea to LLM reasoning tasks for cost-saving, which samples answers from both Chain-of-Thought [303] and Program-of-Thought [60] prompts. Overall, cascade inference is a promising direction for enhanced inference efficiency, but it is still challenging to design an accurate dispatching mechanism to avoid compromising model quality.

3.1.2 Architecture Design. This subsection explores innovative architecture designs tailored for LLMs. Researchers have proposed novel model architectures [129] beyond the original Transformer that strike a balance between model size, performance, and efficiency, opening new avenues for faster and resource-efficient inference.

- **Configuration downsizing:** To reduce the computation cost of LLM inference, a straightforward approach is to downsize the model configurations, such as using shallow encoders [114, 217] or decoders [158], weight sharing, and vocabulary shrinking [267]. However, reducing the number of model parameters also affects the downstream tasks’ performance.

Table 1. Comparisons of Attention Simplification Methods in Prior Efficient Transformers and Recent LLMs

Attention Type	Selective	Sliding + Dilated	Global token	Hash-based
Sparse Pattern				
References	Top-k [126], Sorting [283], Adaptive [70], Informer [366]	Sparse Transformer [65], LongFormer [43]	Star Transformer [124], GMAT [125]	Reformer [166], Routing Transformer [251]
LLM	Scissorhands [199], H ₂ O [355]	Mistral-7B [148],	StreamingLLM [317],	Sparse hash
Applications	[37, 149, 181, 220, 355]	[317], LongNet [82]	Summary [63], Landmark [218]	attention[233]

— **Attention simplification:** One prominent challenge associated with self-attention calculations is the computational complexity $\mathcal{O}(L^2)$, which scales quadratically with the input sequence length L . Numerous Transformer variants [284] have been proposed to simplify the standard attention into more efficient alternatives for very long sequence tasks, such as sparsification [342], kernelization [160], and factorization [298]. Recently, there is a trend of borrowing the ideas from prior attention simplification approaches, generalizing and combining them to shorten the context or reduce the size of KV cache, as well as the attention complexity, with slightly decoding quality degradation (e.g., sliding window attention [148, 353], hash-based attention [62, 233], dilated attention [82], product quantization [349]). One category of these approaches is *context compression* by compressing the context into fewer *soft* tokens (e.g., replacing with summary tokens [63] or landmark tokens [218], leveraging additional autoencoder schemes [108, 198]) or directly dropping or rephrasing unimportant context tokens based on different importance guidance [94, 149, 181, 220] (or called *semantic compression*). For example, adaptively sparse attention [37] takes a learning-based approach to eliminate uninformative context tokens dynamically for each token. Scissorhands [199] and H₂O [355] select a few important tokens that might have a substantial influence for future decoding process and save their KV cache. StreamingLLM [317] values the initial tokens and maintains them with the sliding window, which is also similar to prior work [43]. TriForce [275] extends this idea to reduce the drafting latency and proposes a hierarchical speculative decoding algorithm. FastGen [107] allows different attention heads to employ different emphasizing patterns adaptively. Table 1 illustrates the sparse attention patterns of four representative categories of approaches and their applications. However, due to the incomplete context, these approaches may face inevitable information loss in real workloads with more complex attention distributions.

— **Activation sharing:** Another direction is sharing the intermediate activations to improve the attention calculation efficiency. Attention sharing approaches [182, 310, 318] observe the similarity among different layers' attention matrix distribution and reuse these attention matrices to reduce the computation costs. **Multi-query attention (MQA)** [260] makes different heads share a single set of keys and values to reduce the memory bandwidth requirements in the incremental inference. **Group-query attention (GQA)** [33] relaxes the single set of keys and values restriction to multiple sets and each set is coupled with a group of queries. **Multi-Head Latent Attention (MLA)** [190] jointly compresses keys and values in a low-rank latent vector, leading to significant memory reduction. They have been successfully adopted by several recent public LLMs and shown their superior performance, including, MQA-based models such as Falcon [375], PaLM [68], ChatGLM2-6B [7], GQA-based models

like LLaMA-2 [290] and Mistral-7B [148], and MLA-based models such as DeepSeek V2, V3, and R1 [121].

- **Conditional computing:** The sparsely-activated **Mixture of Experts (MoE)** [71, 261] paradigm partitions a model’s capacity across various “experts”, which are smaller neural networks, each specializing in different subsets of the data. It allows the system to only invoke the necessary experts for a given input based on certain routing mechanisms [92, 176, 226, 250, 258, 369], rather than computing over the entire massive model, yielding computational and memory efficiency [87]. For example, TaskMoE [169] illustrates that task-level routing enables model increase capacity compared with token-level counterpart, while improving the inference throughput. As LLMs continue to grow, the MoE architecture stands out as a promising avenue to ensure both scalability and efficiency for future LLMs. In the meanwhile, the dynamic nature of MoEs also demands special system optimization from both distributed communication [130, 136, 138, 178, 227, 247, 357] and GPU kernel implementation [104, 363] to facilitate MoE inference efficiency.
- **Recurrent unit:** Although **recurrent neural networks (RNN)** (e.g., LSTM [252]) tend to struggle with capturing long-term dependencies in sequences [161], there are still several approaches using recurrent units to replace Transformer modules and achieve linear computational and memory complexity during inference, such as RWKV [237] and RetNet [279]. Specifically, unlike prior approaches, these recent explorations are mostly built on the linear attention (i.e., Linear Transformer [160], Attention Free Transformer [345]) representation. After the reformation, they overcome the $\mathcal{O}(L^2)$ bottleneck of attention by modeling interactions between tokens with linear recurrence units (e.g., state space models [100, 115, 116, 206], LRU [231]), which are easier to maintain parallelizable training property. Their design is also composed of various position encoding modules [273], exponential decay mechanisms [230] and a stack of token-wise non-linear MLPs [288, 339] or GLUs [74] to improve the model representation capability. Recently, they have shown promising results on both model performance and computation efficiency. However, whether recurrent units can successfully replace Transformers for LLMs still remains an open problem (i.e., especially for long sequences).

3.1.3 Model Compression. Here, we delve into techniques for model compression, which aim to reduce the memory footprint and computational requirements of LLMs by creating more efficient and compact models without significant loss in performance.

- **Knowledge Distillation:** One line of work is knowledge distillation, which trains a small student model with the supervision from a large teacher model. Most previous approaches in this direction are exploring white-box distillation [119, 153, 254, 277, 299], which require accessing the entire teacher model parameters. Due to the arising of API-based LLM services (e.g., ChatGPT), several black-box distilled models attract lots of attention, such as Alpaca [282], Vicuna [64], WizardLM [322] and so on [238, 373]. These models usually have fewer model parameters but have shown promising performance on various downstream tasks compared with the original LLMs (e.g., GPT-4 [31]).
- **Network pruning:** Network pruning methods [214, 255, 255] have been extensively studied in the past few years but not all of them can be directly applied to LLMs. It is imperative to take into account the potentially exorbitant computational costs associated with retraining, as well as assess whether the pruning yields discernible gains in inference efficiency based on the underlying system’s implementation. Some recent approaches [90, 170, 203, 256] apply structural pruning methods on LLMs, which removes entire structured LLM components, facilitating efficient GPU speedups. For example, Deja Vu [201] cuts off specific attention

heads and MLP parameters guided by the contextual sparsity hypothesis without modifying pre-trained models. There are also some recent advancements in unstructured methods [42, 96, 276, 294, 325], which usually achieve 50–60% sparsity for LLM compression. It is noteworthy that they can further generalize to semi-structured N:M sparsity (i.e., 2:4 and 4:8) [216], leading to significant inference speedup with NVIDIA sparse tensor cores' acceleration. LoSparse [184] and DSFormer [52] approximate model weights with a small dense and a sparse semi-structured matrix using low-rank factorization. Flash-LLM [315] relaxes this requirement by providing a memory-efficient SpMM implementation for unstructured pruning using tensor cores. PowerInfer [270] assumes skew access of these sparsely-activated neurons and proposes a GPU-CPU hybrid inference engine, making GPU and CPU handle different neurons.

3.2 System Optimization

This section investigates LLM inference system optimization techniques to accelerate LLM inference without modifying the LLM computation semantics. The goal of this line of work is to improve the system efficiency by refining the underlying systems and frameworks used for LLM inference.

3.2.1 Low-bit Quantization. This section explores state-of-the-art low-bit quantization techniques that enable efficient representation of model weights and activations. By using fewer bits (i.e., less than 32) to represent numerical values, these methods significantly reduce memory consumption and accelerate inference on hardware platforms. One line of approach is to quantize LLM, and these quantization methods can be briefly categorized into two directions: **Quantization-Aware Training (QAT)** and **Post-Training Quantization (PTQ)** [331]. PTQ reduces the computational precision of model weights [77, 79, 97, 98, 141, 189] and even activations [316, 332, 340] into either INT8 or INT4 by using custom CUDA kernels [180, 235] or compilations [359] for efficiency benefits, such as W8A16 (i.e., INT8 weight-only quantization and FP16 or BF16 activations), W4A16 in GPTQ [97], W8A8 in SmoothQuant [316] and W4A4 [314]. The evolution of hardware also meets these requirements. One supporting evidence is that NVIDIA's recent architectures like Turing and Ampere have included INT8 and INT4 tensor cores, and the latest Hopper architecture has disabled INT4 support but introduced FP8 tensor cores for better numerical precision (e.g., H100 GPU can reach 60× TFLOPS for FP8 as opposed to FP32).

Existing approaches usually adopt various quantization functions, including uniform methods (i.e., Round-to-Nearest) and non-uniform methods [163]. To relieve the performance loss from low-precision, QAT integrates quantization during model training [78, 200]. It is worth noting that due to challenges in the underlying system implementation, low-precision quantization methods may potentially result in slower inference speeds compared to conventional precision levels such as FP16 [77]. While low-precision methods significantly reduce the resource requirements for model deployment, there is also research indicating that quantization methods can have a notable impact on the model's inference performance due to the presence of scaling laws [80]. In addition, quantization has also been applied to context compression (e.g., CacheGen [198]) and memory-efficient fine-tuning (e.g., QLoRA [78], PEQA [162]), resulting in lower memory consumption for LLM inference.

3.2.2 Parallel Computation. This section examines parallel computation strategies tailored for LLMs. Leveraging parallel processing capabilities of modern hardware architectures, these methods distribute computation across multiple cores or devices, leading to substantial speedup during inference.

- **Model parallelism:** Most model parallelism approaches are first proposed for distributed training of large-scale DNNs, especially for Transformer-based models. For example, tensor model parallelism [269] (TP) splits the model layers (e.g., attention, FFN) into multiple pieces

from internal dimensions (e.g., head, hidden) and deploys each on a separate device (e.g., GPU). It can significantly reduce inference latency through parallel computing, which is widely used for multiple GPUs within the same machine, especially for scenarios with high-speed NVLink connections. PaLM inference [240] extends TP on large-scale Transformer inference by involving 2D tensor parallelism [295] and claims lower theoretical communication complexity for large clusters (more than 256 devices). For MQA with only one head for keys and values, it further involves data parallelism to the hybrid tensor partition strategy. Pipeline model parallelism [145, 223] (PP) arranges the model layers in a sequence across multiple devices. Each device is responsible for a pipeline stage that consists of multiple consecutive model layers. While PP can significantly increase the number of inputs processed per unit of time (throughput), it doesn't inherently decrease the time taken to process a single input from beginning to the end (latency) like TP. Different parallelism techniques introduce varying degrees of communication overhead and computational latency [140]. To achieve optimal performance and resource utilization, automatic parallelism has been widely studied by prior approaches for distributed training (e.g., Alpa [360], FlexFlow [147, 293], Galvatron [213]). By replacing their cost model to fit the predictable runtime of auto-regressive inference of Transformer models like [224], it's easy to apply previous automatic searching algorithms (e.g., dynamic programming, integer linear programming) to LLM serving (e.g., AlpaServe [186], FlexFlow-Serve [12], SpotServe [212]) and determine the most efficient parallelism strategy without manual intervention.

- **Sequence parallelism:** As LLMs increasingly handle tasks requiring extensive context, such as document analysis or conversational agents, processing lengthy sequences efficiently becomes critical. **Sequence parallelism (SP)** distributes the computational and storage load by splitting the processing of long sequences across multiple GPUs along the sequence length dimension. Each GPU processes a portion of the sequence, computes local attention outputs, and then communicates the necessary intermediate results (using techniques such as ring-style [49, 192] or all-gather [142, 168] operations) to ensure that the full sequence context is incorporated into the final output. Meta further explores the implementation details of context parallelism [327] and proposes two ring attention variants for lower latency under varying context lengths and KV cache hit rates. To handling the growing context window, LoongServe [306] introduces elastic SP, automatically scaling-up and scaling-down according to the context length changes.
- **Cloud scaling:** Recent advancements in cloud scaling have significantly enhanced the scalability and cost-effectiveness of serving LLMs. One notable approach involves utilizing preemptible or spot instances—cost-effective cloud resources that can be reclaimed by providers with minimal notice. SpotServe [212] is a distributed LLM serving system designed to operate on preemptible instances, dynamically adjusting parallelization configurations, and migrating GPU context to maintain reliable performance despite potential instance preemptions.

Similarly, SkyServe [204] leverages a combination of spot instances across various regions and clouds to enhance availability and mitigate correlated preemptions. Another innovative direction is the application of serverless computing to LLM serving. ServerlessLLM [103] the cold start challenge by implementing a multitier checkpoint loading system, leveraging underutilized GPU memory and storage to reduce the startup latency. Collectively, these advancements demonstrate the potential of integrating cloud computing paradigms to optimize the scalability, reliability, and cost-efficiency of LLM serving.

- **Decentralized inference:** Serving LLMs in decentralized environments leverages geodistributed or even heterogeneous resources to perform inference tasks. This approach utilizes

idle GPU resources across a network of devices, allowing for efficient processing of LLM workloads without relying on an expensive centralized infrastructure. Decentralized LLM serving addresses challenges such as high costs and limited availability of GPU resources, making it a viable alternative to traditional centralized systems. Inspired by crowdsourced computing, Petals [46] serves a BLOOM-176B model using collaborated commodity GPUs over the Internet. Helix [207] optimizes the model placement and request scheduling under geo-distributed heterogeneous environments with a max-flow-based approach for more affordable LLM serving. Decentralized inference opens up a new direction on unlocking the overlooked consumer-level GPUs for running LLMs, but also suffers from several practical challenges, such as device heterogeneity [152], limited computational and memory capacity [151], low-bandwidth network [47], fault tolerance and privacy protection [281].

3.2.3 Memory Management. Efficient memory management remains at the forefront of challenges in LLM serving, especially given the inherent memory-intensive nature of transformer architectures. With the growing need for long-sequence inference, the memory footprint of the KV cache stands out as a prime optimization target compared with model weights and the necessary workspace for other activations. As the KV cache memory grows and shrinks dynamically and unpredictably during incremental decoding, the naive approach (e.g., FasterTransformer) pre-allocates a contiguous piece of memory with a maximum sequence length assumption. It wastes memory severely for one) input batches with varied request lengths and two) complex decoding scenarios generating multiple output sequences in parallel (e.g., beam search, parallel decoding). vLLM [171] proposes *paged attention* that partitions the KV cache into non-contiguous memory blocks and significantly improves the batch size as well as throughput. vAttention [241] decouples the allocation of virtual and physical memory using the CUDA virtual memory management approach and mitigates fragmentation while hiding the latency cost of on demand memory allocation. LightLLM [21] takes a more granular token-level memory management mechanism to further diminish memory usage. However, the overheads of such fragmented memory managing mechanisms pose new challenges. Especially for cases where other optimizations are employed to boost the batch size, these fine-grained memory management methods might offer only marginal throughput benefits while substantially amplifying the inference latency. It's evident that memory reduction in LLM inference is intricately tied with other algorithmic innovations and system-level optimizations. While some might work well for specific workloads, they might counteract one another, leading to a degraded overall performance. Striking the right balance between memory efficiency and computational performance of LLM inference systems remains an open and pressing challenge in the field.

There are also some approaches [35, 36, 123, 211, 266] enabling offloading techniques to use larger but slower memory (e.g., CPU DRAM, SSD) to save model parameters or KV cache in addition to the limited device memory (e.g., GPU DRAM). For instance, systems like InfiniGen [174] and HCache [106] dynamically manage and offload KV cache to external memory tiers, thereby reducing GPU memory pressure and accelerating state restoration. Similarly, architectures such as Pensieve [338] and Mooncake [244] disaggregate the inference pipeline to enable efficient migration and restoration of model states, while methods like CachedAttention [105] exploit offloading to reuse attention computations across multi-turn conversations, striking a balance between performance and cost-efficiency in LLM serving.

3.2.4 Request Scheduling. Efficiently scheduling incoming inference requests is crucial for optimizing LLM serving. This section reviews request scheduling algorithms that maximize resource utilization, guarantee response time within latency **service level objective (SLO)**, and handle varying request loads effectively. Request scheduling for LLM serving shares commonalities with

general ML serving techniques, as both aim to efficiently manage incoming requests and optimize resource utilization. These common aspects include dynamic batching [34], preemption [128], priority [225], fairness [265], swapping [41], model selection [120], cost efficiency [348], load balancing, and resource allocation [304]. However, LLM serving also introduces unique challenges due to its distinctive characteristics, such as the massive model size, iterative autoregressive decoding mechanism, unknown variable output length and state management for context information.

Early LLM serving systems (e.g., FasterTransformer over NVIDIA Triton) only support request-level scheduling which is similar to prior approaches. Orca [337] first notices the gap between generative LLMs and the request-level scheduling of previous ML inference systems. Considering the variable output sequence length, it schedules the execution of the engine at the granularity of iteration with a **first-come-first-serve (FCFS)** order and enables batching a selected set of operations for better hardware utilization. Plenty of following approaches inherit the *selective-batching* and *iteration-level scheduling* policy, such as *continuous batching* in vLLM and RayLLM [27] and *in-flight batching* in TensorRT-LLM [25]. Moreover, SpecInfer extends to speculative decoding by iteratively selecting a batch of requests to perform one iteration of speculative inference and verification. FastServe [307] concentrates on the **job completion time (JCT)** and involves iteration-level preemption to prioritize requests with shorter input length, instead of FCFS. Sarathi-Serve [32] targets the pipeline bubbles in distributed inference caused by the initial iteration of varying length input requests. To saturate the GPU compute, it splits the input prompts into uniform chunks and piggybacks the chunk slot with other requests' decoding iterations if possible, which is also adopted by DeepSpeed-FastGen called Dynamic SplitFuse [9]. S³ [155] involves an output sequence length predictor and helps to schedule more concurrent requests within the GPU memory constraint for larger batch size and higher inference throughput. The prefill-decode disaggregation scheme was first proposed by Splitwise [236] to split prefill from decode into different GPUs, while some concurrent work (e.g., DistServe [364], ExeGPT [228]) have proposed similar architectures. LLM microservicing [154] introduces a request-level programmable router to support dynamic reconfiguration of multiple disaggregation orchestration strategies.

Mixed LLM serving workload often makes request scheduling more complex. For example, TetriInfer [135] identifies the interference issue when serving heterogeneous downstream workloads (e.g., conversation, summarization, writing) with different length distribution. To address this problem, it utilizes the chunked prefill technique, applies the disaggregated architecture, and incorporates a two-level scheduling algorithm augmented with a length prediction model. Andes [194] considers users' diverse Quality-of-Experience metrics for LLM-based text streaming services and introduces a preemptive request scheduling approach. ConServe [243] executes online and offline LLM inference tasks simultaneously and leverages a priority-based scheduler to improve the overall resource utilization. Llumnix [274] reschedules the heterogeneous requests react to the unpredictable workload dynamics at runtime based on a load-balancing scheduling policy and implements on top of inference engines.

3.2.5 Kernel Optimization. In this subsection, we delve into kernel-level optimizations, which target the performance of specific operations within the language model inference pipeline. These optimizations leverage hardware-specific features and software techniques to accelerate critical computation kernels.

- **Kernel fusion:** To reduce overheads from kernel launching and memory accessing, kernel fusion is widely adapted by previous DNN frameworks and compilers. Since the backward computation is not required for LLM inference, more kernel fusion chances exist. Several contemporary Transformer inference engines (e.g., FasterTransformer [2], TenTrans [309], TurboTransformers [91], LightSeq [300], ByteTransformer [346]) and compilers (e.g. Welder

[268]) propose to fuse (1) GEMMs with the same shape (e.g., the three linear transformations for query, key, and value), and (2) Add Bias with the other non-GEMM kernels, such as residual connection, layer normalization and activation functions (e.g., ReLU). Among these, the optimization of fused MHA kernel has been extensively explored and will be discussed in the following aspect.

- **Tailored attention:** To make the attention operations run efficiently on a GPU, customizing or tailoring the GPU kernels specifically for the attention calculation is crucial. For example, cuDNN has provided a fused MHA kernel API [23]. Meanwhile, several implementations have been open-sourced for more performance gains. These can be roughly classified into two categories due to the special autoregressive decoding mechanism. One is for the first iteration (i.e., the initial/prefill/context/prompt phase), which processes all tokens from the input prompt in parallel. For example, xFormers [175] extends the online softmax trick [67, 215, 246] to the whole attention calculation using CUTLASS [24]. The other is for the following iterations (i.e., the incremental/decode/generation phase) and the kernel only generates one output token per iteration. For autoregressive decoding, a common practice is to save the previously computed keys and values so that only a single query is required to compute when generating a new token instead of rerunning the entire sequence. The main direction of optimizations in this field is maximizing thread occupancy and minimizing the on-device HBM access (i.e., using shared memory or registers [58]). They usually parallelize across the batch size and number of heads dimension (e.g., FasterTransformer) to distribute workloads. Some further enable parallelizing the sequence length dimension by partitioning the KV cache into chunks but require reducing the chunk-wise results at last, such as FlashDecoding [292]. A subsequent work FlashDecoding++ [133] removes such synchronization for partial softmax by introducing a unified maximum value known in advance. It is necessary to select the appropriate parallel dimension based on the workloads for better thread utilization. FlashInfer [334] utilizes the block-sparse format to unify diverse KV-Cache patterns, provides a customizable attention template to support different attention variants, and leverages CUDAGraph to maximize GPU utilization.
- **Variable sequence length:** Another unique challenge of LLM inference is that the sequences can vary in both input length and output length, and the latter is unknown in advance. One way to speed up inference is to process multiple sequences in a batch at once (Section 3.2.4). However, when a batch of sequences has variable input lengths, padding is often used to make them all the same length for batch processing, wasting computational and memory resources. To alleviate some of these inefficiencies, various strategies can be employed. Packing technique [1, 346] stores the sequences into a continuous memory space without padding and only unpacks before attention calculation. Ragged tensor [93] further supports computation with minimal padding using compiler-generated kernels. Bucketing the sequence into a smaller computation granularity (e.g., chunks [86]) is also a possible solution to alleviate memory usage of padding tokens. Due to the mixed execution of the initial phase and incremental phase, bucketing input prompts [32] also brings new challenges to the memory management and request scheduling (Section 3.2.4).
- **Automatic compilation:** Most existing LLM inference systems utilize vendor-specific libraries as their backend, such as cuBLAS, cuDNN, and CUTLASS, which provide optimized kernel implementations. To further improve the inference efficiency, they also take great efforts on optimizing manually-written kernels for specific LLM operators (e.g., attention) over NVIDIA GPUs. Despite of these work, the trend of using automated DNN compilers still exists, such as TVM (i.e., Unity [253], Relax [172] and TensorIR [95, 335]), MLIR [159], JAX [99], OpenAI Triton [287], TASO [146], Mirage [311], and TorchInductor [312]. The

Table 2. Comparison of state-of-the-art Open-Sourced GPU-Based LLM Serving Systems

Name Github Ref.	Parallel Computation			Iteration- Sche.	Attention Kernel		Prioritized Opt. Target		Main Features
	TP	PP	Offload		Initial	Incremental	L_{at}	T_{pt}	
FasterTransformer [2]	✓	✓			cuBLAS GEMM	Fused attention	✓		- Manually-written kernel - Lightweight runtime
FlexFlow-Serve [12]	✓	✓	✓	✓	cuBLAS GEMM	Tree attention	✓		- SpecInfer [211] - Extremely low L_{at}
vLLM [29]	✓		✓	✓	xFormers	Paged attention		✓	- Block-level KV cache [171] - Enlarging batch size & T_{pt}
FlexGen [13]		✓	✓		torch. bmm	torch. bmm		✓	- CPU&Disk Offload [266] - Maximizing single GPU T_{pt}
TGI [18]	✓			✓	Flash attention	Paged attention		✓	Huggingface integration
DeepSpeed-Inference [3]	✓	✓			cuBLAS GEMM	cuBLAS GEMM	✓		- Kernel auto-injection [10] - Multi-GPU & Multi-Node
ZeRO-Inference [3]	✓	✓	✓		cuBLAS GEMM	cuBLAS GEMM		✓	- CPU&NVMe Offload [36] - Maximizing single GPU T_{pt}
Light-LLM [21]	✓			✓	Flash attention	Token attention		✓	- Token-level KV cache - Enlarging batch size & T_{pt}
MLC-LLM [285]	✓			✓	compiled MatMul	Paged attention	✓		- Universal deployment - Multiple types of GPUs
TensorRT-LLM [25]	✓	✓	✓	✓	cuBLAS/ Flash-attn	Paged attention	✓		- NVIDIA Triton integration - Rich features supported

compilation approach can help discover potentially more efficient operator implementations (e.g., expression derivation [361]), and more importantly, facilitate adaptation to alternative hardware platforms, including mobile and edge devices, CPUs, DL accelerators, and other types of GPUs (e.g., AMD GPUs and Apple M2 Ultra).

In summary, our taxonomy categorizes the advancements in efficient LLM serving into two key dimensions: algorithmic innovations and system optimizations. Both dimensions play a critical role in reducing inference latency and enhancing throughput, though they approach these goals from different angles. The algorithmic side techniques streamline the generation process to accelerate inference while carefully managing the balance between speed and accuracy. Similarly, system-level optimizations refine the underlying computational framework to optimize resource utilization and further boost performance. When deploying these techniques in real-world applications, trade-offs between efficiency and accuracy must be carefully considered. For example, in real-time conversational systems may tolerate slight accuracy losses by using low-bit quantization for faster response times, whereas high-stakes applications like medical diagnostics demand the precision afforded by more computationally intensive methods.

4 Software Frameworks

Generative LLM serving requires a full stack of optimizations and many recent works have started to develop software frameworks to provide efficient LLM inference deployment service. In the following, we revisit these systems and investigate a comprehensive analysis of several representative open-sourced GPU-based LLM serving systems in Table 2. The analysis does not contain some popular related projects, including (1) specialized solutions for other hardware (e.g., PopTransformer [17], CTranslate2 [8], lammap.cpp and ggml [14]), and (2) deployment solutions built on top of the other systems, like OpenLLM [26] (vLLM), Xinference [30] (ggml + vLLM + xFormers), LMDeploy

[20] (FasterTransformer), gpt-fast [15] (PyTorch), DeepSpeed-MII and DeepSpeed-FastGen [11] (DeepSpeed-Inference), and RayLLM and RayServe [27] (vLLM).

We compare these state-of-the-art LLM serving systems and summarize their differences in several aspects. First, most of these systems support tensor parallelism to enable multi-GPU inference and improve the system performance. And some of them future support pipeline parallelism or offloading to support inference over multi-node or resource-constrained environments individually. Second, partial systems learn from Orca and implement the iteration-level scheduling. Third, we investigate the attention kernels of these systems and introduce their implementations in terms of the initial and incremental phases, respectively. For the initial phase, they usually adapt a batched **general matrix multiply (GEMM)** approach (e.g., cuBLAS, torch, and Relay) and some utilize the online softmax trick to reduce HBM access (e.g., Flash-attention, xFormers). The incremental phase is more challenging because the per-token generation scheme results in lower computational intensity. To improve the GPU utilization, FasterTransformer manually fuses the attention calculations (e.g., linear projection, positional bias, dot product, softmax, etc) into a single high-performance kernel template and involves several kernel optimization techniques, such as caching with shard memory, warp-shuffle instruction for reduction, **half matrix multiplication and accumulation (HMMA)** with tensor core and multiple-precision support. FlexFlow-Serve enables speculative decoding and provides a tree-based parallel decoding kernel to verify the speculated tokens from multiple sequences (i.e., from multiple small models or different beams or parallel sampling) with zero-memory redundancy and maximum thread parallelism. vLLM extends the fused **multi-head attention (MHA)** kernel from FasterTransformer by partitioning the KV cache into pages to eliminate redundant memory usage, especially for parallel sampling scenarios. LightLLM takes a follow-up approach by partitioning the KV cache into more fine-grained token-wise pieces.

Note that, there still remain some other notable aspects that are not covered by the above discussions. For example, even for the most popular Flash and Paged attention kernels, they are usually implemented in different ways across these systems. TGI directly imports the original Flash/Paged attention libraries, LightLLM adopts kernels implemented by OpenAI Triton, MLC-LLM generates kernels by TVM, and TensorRT-LLM modifies from FasterTransformer's fused attention kernel to support paged attention. Another example is about the input-aware kernel selection. For the initial phase, TensorRT-LLM selects from cuBLAS and Flash attention based on the context length. Besides the attention calculation, for the linear projection operators, there is also a recent trend of replacing GEMM with **general matrix-vector product (GEMV)** to handle the cases of small batch size (i.e., one) more efficiently. And these systems also have many other different features, such as programming language (i.e., C++, Python), low-precision support (i.e., FP16, INT8), supported hardware and models. In summary, these different choices of design and implementation are largely determined by their prioritized optimization target. For example, vLLM proposes paged attention to improve the batch size for higher *throughput* (T_{pt}), while FlexFlow-Serve leverages SpecInfer to accelerate decoding for lower *latency* (L_{at}). Basically, low latency and high throughput are dual optimization targets in LLM serving systems, representing complementary but often conflicting objectives, necessitating a balanced strategy to optimize the trade-off between rapid response for individual tasks and maximizing the volume of tasks processed over a specified time frame. Some recent studies [73] further decompose the response latency by $TTFT + TPOT \times \text{output sequence length}$, where TTFT represents *Time To First Token* and TPOT represents *Time Per Output Token*. The former is driven by the initial phase processing speed while the latter directly depends on per-iteration execution time during incremental decoding. Distinguishing these two metrics is beneficial to LLM service providers, leading to different system design choices and user experience (e.g., faster application responsiveness [198], and longer prompts [9]). Some recent kernel libraries and compilers (e.g., FlashInfer [334], Mirage [311], and FlexAttention [83]) provide more flexible

program interfaces and generate optimized kernels adaptively according to the input configurations, which are also integrated with mainstream LLM serving engines. Although it unlikely to have a one-size-fits-all solution, we believe that future LLM serving systems will continually integrate these differentiated features, thereby continuously improving system efficiency and hardware utilization.

5 Benchmarks

Most existing work evaluate their system performance under real-world traces by leveraging the dynamic request arrival patterns from public production datasets (e.g., BurstGPT [302] and Azure [236].) Building a comprehensive and reproducible benchmark for comparing the performance of various LLM serving system like MLPerf [249] is a critical endeavor for both academic and industrial communities in this field. It will not only help LLM users select the right system solutions but also encourage researchers and developers to keep pace with the advanced optimizations. Unfortunately, despite of some prior reports [5, 22], up to this point, the community has not yet launched a convincing enough benchmark that takes into account all influencing factors. This is mainly because of the numerous evaluation settings, including model configuration, hardware environment, and request load, among others. Most related benchmarks (e.g., LLM-Perf Leaderboard [139]) are designed for comparing the performance of different LLMs under the same infrastructure. LLM-Inference-Bench [66] has been proposed to evaluate the hardware inference performance of LLMs, including GPUs from NVIDIA and AMD and specialized AI accelerators, Intel Habana and SambaNova. Testing under a limited number of setting combinations cannot yield conclusions with credibility. For example, certain system optimization techniques can only achieve performance advantages under high or low load conditions, and conversely, they might even be detrimental. Besides, when measuring inference latency, how to exclude additional overheads not related to GPU inference (such as request scheduling overhead, inherent network latency, etc.) due to differences in system design is also a challenging topic. Additionally, a fair benchmark test needs to consider the strict alignment of model output content, which is often overlooked in many tests.

Furthermore, designing such benchmarks demands careful calibration of trade-offs among throughput, latency, and cost-efficiency across a spectrum of workload scenarios—from bursty and unpredictable traffic to sustained high-load conditions. A truly comprehensive evaluation must incorporate varied request patterns, diverse hardware configurations, and detailed measurements that isolate core inference performance from extraneous system overheads. Moreover, it is essential to account for differences in resource allocation, scheduling strategies, and communication latencies, ensuring that any performance gains are attributable to the underlying optimizations rather than artifacts of the testing environment. Establishing such a standard benchmark will provide valuable guidance for both practitioners and researchers, facilitating transparent comparisons and driving further innovation in LLM serving systems.

6 Connection with other surveys

Our survey on efficient generative LLM serving and inference complements and extends the scope of existing literature in the field, while maintaining a distinct focus. Among the related works, [164] comes closest in subject matter exploring the design of more general Transformer models and domain-specific accelerators. However, our survey differentiates itself by focusing specifically on generative LLM serving, a nuanced area that has not been the central focus of other studies. Moreover, some studies delve into experimental investigations of LLM inference efficiency on GPUs [224, 351] and novel accelerators [88], offering valuable empirical insights that are directly relevant to our focus on serving efficiency. Additionally, LLMCarbon [89] addresses an increasingly important aspect of LLM deployment—its environmental impact (e.g., carbon footprints). While our

survey's primary focus is efficiency from a performance standpoint, the environmental lens provided by such studies is undeniably relevant and respected in our broader discussion. Some surveys and benchmarks [143] offer valuable insights into model compression [127, 291, 374, 374] and quantization [112, 331]. These studies lay a groundwork that indirectly supports our exploration of related directions. Some studies [72, 221] provide essential context for understanding LLM effectiveness (e.g., accuracy, perplexity, factuality and so on), which is beyond the scope of this survey. Our survey also acknowledges the contributions of prior surveys [44, 205] focusing on distributed training of large-scale DNN models, as they inform the backdrop against which LLM serving must be considered. In essence, our survey situates itself amidst a diverse array of studies, drawing from and contributing to a more holistic understanding of LLM serving efficiency, including both algorithmic innovations and system optimizations. By integrating insights from these various areas, we aim to provide a nuanced and comprehensive overview of the latest advancements and challenges in the field.

7 Future Direction

As we stand at the forefront of LLM advancements, it becomes increasingly important to not only understand the current state of these technologies but also to anticipate and shape their future trajectory. Particularly in the realm of generative LLM serving, there is a vast landscape of unexplored possibilities and emerging challenges. The rapid evolution of this field necessitates a forward-looking approach, where identifying potential avenues for innovation and improvement is crucial. This foresight not only prepares us to adapt to upcoming technological shifts but also guides the research community toward addressing the most pertinent and impactful areas. In this context, we outline several promising directions for future research and development, each offering the potential to significantly enhance the efficiency of serving generative LLMs.

Development and Enhancement of Hardware Accelerators Future progress in enhancing generative LLM serving efficiency could be significantly driven by the development and refinement of specialized hardware accelerators, complemented by a co-design approach that aligns hardware and software optimizations. For instance, integrating memory closer to processing units or optimizing chip architectures to better align with the data flow of LLM algorithms can lead to substantial reductions in latency and energy consumption. This approach has been exemplified in recent GPU advancements, like NVIDIA's Hopper architecture [4], which demonstrates improvements in HBM and SRAM capacity, memory bandwidth, computing units and bisection bandwidth, directly benefiting the processing of LLMs. Continued innovation in this area could involve designing hardware, that is, inherently tuned to the computational patterns of generative LLMs, such as optimizing for the specific demands of attention mechanisms and tensor operations that are prevalent in these models, eventually influencing the design and implementation of LLM serving systems.

Efficient and Effective Decoding Algorithms The development of more efficient decoding algorithms could substantially improve serving efficiency. Motivated by the demand for more resource-efficient ways to utilize the vast knowledge encapsulated within LLMs, future work could explore alternative approaches to the traditional auto-regressive methods and unlock the generation speed for real-time applications while maintaining the decoding quality. One promising direction is *generalized speculative inference* as it enables preserving the same generation quality. Specifically, the small speculative model can be generalized to any other forms of methods that can generate draft tokens more efficiently than LLMs, such as knowledge retriever and user-defined functions [211, 328]. For example, some subsequent works arose recently, replacing the draft model with early exiting [39, 134, 330, 350] or non-autoregressive decoding [101, 109]. Some further generalize speculative decoding to handle other types of workloads, such as retrieval-augmented generation [356], long input sequences [193, 329], and diffusion models [69, 144]. In summary, the development

of efficient decoding algorithms like speculative decoding coupled with the underlying system optimizations represents a significant opportunity to enhance the serving efficiency of generative LLMs.

Long Context/Sequence Scenarios Optimization As the application of LLMs continues to expand into more sophisticated scenarios, the demand for processing longer contexts or sequences is steadily growing. Serving LLMs with long-sequence workloads requires resolving the challenges from both the algorithm and system sides. In terms of LLMs, they often suffer from *length generalization failure* when sequences get longer than what was observed during training [242] even enabling relative positional encoding [59] or after fine-tuning on longer corpora [40]. Even for some models that claim to support ultra-long contexts, studies have found that they encounter a situation of “loss in the middle” [195]. Current approaches attempt to alleviate such limitations by reducing the computational sequence length while preserving relevant information, such as retrieval augmentation [324], sequence compression [150] and caching [113]. For the LLM serving systems, longer sequence brings critical challenges, including more memory consumption and access of KV cache and quadratic increasing computational complexity of self-attention.

Investigating Alternative Architectures Although Transformer models and self-attention mechanisms currently dominate the landscape of LLMs, exploring alternative architectures is a promising direction for future research. The field of DL has historically seen a constant alternation of dominant architectures, with each new paradigm shift bringing about significant advancements. Given this trend, it’s important to consider other architectural approaches that could offer distinct advantages, especially for improved computational efficiency. For instance, some recent studies explore *attention-free* methods [48], using pure **Multi-Layer Perceptron (MLP)** architectures to replace attention mechanisms. These changes also bring new innovation opportunities of the underlying inference engine, such as KV cache management [234]. The evolution of DNN model architecture is not only a natural progression, but also a necessary exploration to uncover more efficient and effective ways of structuring LLMs.

Exploration of Deployment in Complex Environments As the application of LLMs expands, a crucial future direction involves exploring and optimizing their deployment across various complex environments. This exploration goes beyond traditional cloud-based deployments to include scenarios like edge computing, hybrid computing (combining cloud and edge computing), decentralized computing, and the utilization of more affordable resources like spot instances. Each of these environments presents unique challenges and opportunities for LLM serving. For instance, edge computing allows for faster response times and reduced bandwidth usage by processing data closer to the source, but it also poses challenges in terms of limited computational resources and storage capacity. Hybrid computing [245] offers a balanced approach but requires advanced management to distribute computational tasks efficiently. Decentralized computing presents a promising avenue for crowdsourcing computational resources, but it also brings additional considerations regarding data privacy and security [202, 352]. LLM serving over preemptive resources [212] can significantly reduce monetary costs but requires fault tolerance mechanisms to handle their inherent unpredictability and variability, ensuring consistent performance and system reliability. Successfully navigating the challenges from these complex environments will be key for more robust, scalable, and efficient LLM applications.

Automatic Adaptation to Specific Requirements The diverse application-specific requirements create a wide range of innovative LLM serving optimization opportunities, such as parameter-efficient fine-tuning [55, 210, 264, 308, 371], reinforcement or online learning and knowledge updates [208, 263, 365], retrieval from external vector storage [45], multi-round conversation [188], reasoning and planning [102], multi-modal workloads, structured generation [85, 362], and chaining together different LLMs’ capabilities [313] (e.g., multi-agent simulation [320]). These

unique challenges also demand automatic and smooth integration of LLM serving techniques into existing IT infrastructures by extending the optimization space to the whole LLM lifetime, including data acquisition and processing [209], AutoML [289] and model management [222], resource allocations, and performance monitoring.

8 Conclusion

Efficient LLM serving is a fundamental step towards democratizing access to advanced AI technologies. This survey aims to provide researchers, practitioners, and developers with a comprehensive understanding of the existing methodologies, enabling them to make informed decisions when deploying LLMs in real-world environments. By consolidating the latest research findings on algorithms and systems, this survey paper hopes to accelerate progress and foster innovation in the pursuit of highly efficient LLM serving solutions.

References

- [1] 2020. NVIDIA Effective Transformer. Retrieved November 25, 2023 from https://github.com/bytedance/effective_transformer. (2020). Commit: e406421.
- [2] 2021. NVIDIA FasterTransformer. Retrieved November 25, 2023 from <https://github.com/NVIDIA/FasterTransformer>. (2021). Commit: df4a753.
- [3] 2022. DeepSpeed Inference. Retrieved November 25, 2023 from <https://github.com/microsoft/DeepSpeed>. (2022). Commit: 2afa1c7.
- [4] 2022. NVIDIA H100 Tensor Core GPU Architecture. Retrieved November 25, 2023 from <https://resources.nvidia.com/en-us-tensor-core/gtc22-whitepaper-hopper>. (2022).
- [5] 2023. AnyScale LLMPerf leaderboard. Retrieved December 23, 2023 from <https://github.com/ray-project/llmperf-leaderboard>. (2023).
- [6] 2023. AWS Inferentia. Retrieved from <https://aws.amazon.com/blogs/machine-learning/deploy-large-language-models-on-aws-inferentia2-using-large-model-inference-containers/>. (2023).
- [7] 2023. ChatGLM2-6B. Retrieved from <https://huggingface.co/THUDM/chatglm2-6b>. (2023).
- [8] 2023. CTranslate2. Retrieved November 25, 2023 from <https://github.com/OpenNMT/CTranslate2>. (2023). Commit: d963499.
- [9] 2023. DeepSpeed-FastGen. Retrieved November 25, 2023 from <https://github.com/microsoft/DeepSpeed/tree/master/blogs/deepspeed-fastgen>. (2023).
- [10] 2023. DeepSpeed-Inference v.s. ZeRO-Inference. Retrieved November 25, 2023 from <https://github.com/microsoft/DeepSpeed/issues/4234>. (2023).
- [11] 2023. DeepSpeed-MII. Retrieved November 25, 2023 from <https://github.com/microsoft/DeepSpeed-MII>. (2023). Commit: f34b772.
- [12] 2023. FlexFlow-Serve. Retrieved November 25, 2023 from <https://github.com/Flexflow/FlexFlow/tree/inference>. (2023). Commit: 672cdad.
- [13] 2023. FlexGen. Retrieved November 25, 2023 from <https://github.com/FMInference/FlexGen>. (2023). Commit: d34f7b4.
- [14] 2023. ggml. Retrieved November 25, 2023 from <https://github.com/ggml/ggml>. (2023). Commit: a5e4560.
- [15] 2023. gpt-fast. Retrieved December 23, 2023 from <https://github.com/pytorch-labs/gpt-fast>. (2023). Commit: 8c8c463.
- [16] 2023. Graphcore. Retrieved from <https://www.graphcore.ai/posts/dolly-2.0-open-source-language-model-with-chatgpt-like-interactivity>. (2023).
- [17] 2023. Graphcore PopTransformer. Retrieved November 25, 2023 from <https://github.com/graphcore/PopTransformer>. (2023). Commit: 1314598.
- [18] 2023. Huggingface Text Generation Inference. Retrieved November 25, 2023 from <https://github.com/huggingface/text-generation-inference>. (2023). Commit: 3c02262.
- [19] 2023. Intel Extension for Transformers. Retrieved December 23, 2023 from <https://github.com/intel/intel-extension-for-transformers>. (2023). Commit: 37d4007.
- [20] 2023. InterLM LMDeploy. Retrieved November 25, 2023 from <https://github.com/InterLM/lmdeploy>. (2023). Commit: c07f60f.
- [21] 2023. LightLLM. Retrieved November 25, 2023 from <https://github.com/ModelTC/lightllm>. (2023). Commit: 84671a7.
- [22] 2023. Llama-v2-7b benchmark. Retrieved November 25, 2023 from https://hamel.dev/notes/llm/inference/03_inference.html. (2023).
- [23] 2023. NVIDIA cuDNN MultiHeadAttn. Retrieved November 25, 2023 from <https://docs.nvidia.com/deeplearning/cudnn/api/index.html#cudnnMultiHeadAttnForward>. (2023).

- [24] 2023. NVIDIA CUTLASS. Retrieved November 25, 2023 from <https://github.com/NVIDIA/cutlass>. (2023). Commit: b5d8a5d.
- [25] 2023. NVIDIA TensorRT-LLM. Retrieved November 25, 2023 from <https://github.com/NVIDIA/TensorRT-LLM>. (2023). Commit: 6837c81.
- [26] 2023. OpenLLM. Retrieved November 25, 2023 from <https://github.com/bentoml/OpenLLM>. (2023). Commit: b4ea4b3.
- [27] 2023. RayLLM. Retrieved November 25, 2023 from <https://github.com/ray-project/ray-llm>. (2023). Commit: fa3a766.
- [28] 2023. Sambanova. Retrieved from <https://sambanova.ai/press/sambanova-unveils-new-chip-the-sn40l/>. (2023).
- [29] 2023. vLLM. Retrieved November 25, 2023 from <https://github.com/vllm-project/vllm>. (2023). Commit: 7c60044.
- [30] 2023. Xorbits Inference (Xinference). Retrieved November 25, 2023 from <https://github.com/xorbitsai/inference>. (2023). Commit: 22732d8.
- [31] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altmenschmidt, Sam Altman, Shyamal Anadkat, 2023. Gpt-4 technical report. arXiv:2303.08774. Retrieved from <https://arxiv.org/abs/2303.08774> (2023).
- [32] Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav Gulavani, Alexey Tumanov, and Ramachandran Ramjee. 2024. Taming throughput-latency tradeoff in LLM inference with Sarathi-Serve. In *Proceedings of the OSDI 2024*. 117–134.
- [33] Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. 2023. GQA: Training generalized multi-query transformer models from multi-head checkpoints. In *Proceedings of the EMNLP 2023*.
- [34] Ahsan Ali, Riccardo Pinciroli, Feng Yan, and Evgenia Smirni. 2020. Batch: Machine learning inference serving on serverless platforms with adaptive batching. In *Proceedings of the SC 2020*. 1–15.
- [35] Keivan Alizadeh, Iman Mirzadeh, Dmitry Belenko, Karen Khatamifard, Minsik Cho, Carlo C. Del Mundo, Mohammad Rastegari, and Mehrdad Farajtabar. 2023. LLM in a flash: Efficient large language model inference with limited memory. arXiv:2312.11514. Retrieved from <https://arxiv.org/abs/2312.11514> (2023).
- [36] Reza Yazdani Aminabadi, Samyam Rajbhandari, Minjia Zhang, Ammar Ahmad Awan, Cheng Li, Du Li, Elton Zheng, Jeff Rasley, Shaden Smith, Olatunji Ruwase, and Yuxiong He. 2022. DeepSpeed inference: Enabling efficient inference of transformer models at unprecedented scale. arXiv:2207.00032. Retrieved from <https://arxiv.org/abs/2207.00032>. (2022).
- [37] Sotiris Anagnostidis, Dario Pavlo, Luca Biggio, Lorenzo Noci, Aurelien Lucchi, and Thomas Hoffmann. 2023. Dynamic context pruning for efficient and interpretable autoregressive transformers. arXiv:2305.15805. Retrieved from <https://arxiv.org/abs/2305.15805>. (2023).
- [38] Zachary Ankner, Rishab Parthasarathy, Aniruddha Nrusimha, Christopher Rinard, Jonathan Ragan-Kelley, and William Brandon. 2024. Hydra: Sequentially-dependent draft heads for medusa decoding. In *Proceedings of the COLM 2024*.
- [39] Sangmin Bae, Jongwoo Ko, Hwanjun Song, and Se-Young Yun. 2023. Fast and robust early-exiting framework for autoregressive language models with synchronized parallel decoding. arXiv:2310.05424. Retrieved from <https://arxiv.org/abs/2310.05424>. (2023).
- [40] Yushi Bai, Xin Lv, Jiajie Zhang, Hongchang Lyu, Jiankai Tang, Zhidian Huang, Zhengxiao Du, Xiao Liu, Aohan Zeng, Lei Hou, Yuxiao Dong, Jie Tang, and Juanzi Li. 2024. LongBench: A bilingual, multitask benchmark for long context understanding. In *Proceedings of the ACL 2024*.
- [41] Zhihao Bai, Zhen Zhang, Yibo Zhu, and Xin Jin. 2020. PipeSwitch: Fast pipelined context switching for deep learning applications. In *Proceedings of the OSDI 2020*. 499–514.
- [42] Peter Belcak and Roger Wattenhofer. 2023. Exponentially faster language modelling. arXiv:2311.10770. Retrieved from <https://arxiv.org/abs/2311.10770>. (2023).
- [43] Iz Beltagy, Matthew E. Peters, and Arman Cohan. 2020. Longformer: The long-document transformer. arXiv:2004.05150. Retrieved from <https://arxiv.org/abs/2004.05150>. (2020).
- [44] Tal Ben-Nun and Torsten Hoeft. 2019. Demystifying parallel and distributed deep learning: An in-depth concurrency analysis. *ACM Computing Surveys (CSUR)* (2019), 1–43.
- [45] Sebastian Borgeaud, Arthur Mensch, Jordan Hoffmann, Trevor Cai, Eliza Rutherford, Katie Millican, George Bm Van Den Driessche, Jean-Baptiste Lespiau, Bogdan Damoc, Aidan Clark, Diego de Las Casas, Aurelia Guy, Jacob Menick, Roman Ring, Tom Hennigan, Saffron Huang, Loren Maggiore, Chris Jones, Albin Cassirer, Andy Brock, Michela Paganini, Geoffrey Irving, Oriol Vinyals, Simon Osindero, Karen Simonyan, Jack W. Rae, Erich Elsen, and Laurent Sifre. 2022. Improving language models by retrieving from trillions of tokens. In *Proceedings of the ICML*. 2206–2240.
- [46] Alexander Borzunov, Dmitry Baranchuk, Tim Dettmers, Max Ryabinin, Younes Belkada, Artem Chumachenko, Pavel Samygin, and Colin Raffel. 2023. Petals: Collaborative inference and fine-tuning of large models. In *Proceedings of the ACL 2023*.
- [47] Alexander Borzunov, Max Ryabinin, Artem Chumachenko, Dmitry Baranchuk, Tim Dettmers, Younes Belkada, Pavel Samygin, and Colin Raffel. 2023. Distributed inference and fine-tuning of large language models over the internet. arXiv:2312.08361. Retrieved from <https://arxiv.org/abs/2312.08361>. (2023).

- [48] Vukasin Bozic, Danilo Dordevic, Daniele Coppola, and Joseph Thommes. 2023. Rethinking attention: Exploring shallow feed-forward neural networks as an alternative to attention layers in transformers. arXiv:2311.10642. Retrieved from <https://arxiv.org/abs/2311.10642>. (2023).
- [49] William Brandon, Aniruddha Nrusimha, Kevin Qian, Zachary Ankner, Tian Jin, Zhiye Song, and Jonathan Ragan-Kelley. 2023. Striped attention: Faster ring attention for causal transformers. arXiv:2311.09431. Retrieved from <https://arxiv.org/abs/2311.09431>. (2023).
- [50] F. Warren Burton. 1985. Speculative computation, parallelism, and functional programming. *IEEE Transactions on Computers* (1985), 1190–1193.
- [51] Tianle Cai, Yuhong Li, Zhengyang Geng, Hongwu Peng, and Tri Dao. 2023. Medusa: Simple framework for accelerating llm generation with multiple decoding heads. Retrieved November 25, 2023 from <https://github.com/FasterDecoding/Medusa>. (2023). Commit: dd9c8a5.
- [52] Rahul Chand, Yashoteja Prabhu, and Pratyush Kumar. 2023. DSFormer: Effective compression of text-transformers by dense-sparse weight factorization. arXiv:2312.13211. Retrieved from <https://arxiv.org/abs/2312.13211>. (2023).
- [53] Carol Chen. 2022. Transformer Inference Arithmetic. Retrieved November 25, 2023 from <https://kipp.ly/blog/transformer-inference-arithmetic/>. (2022).
- [54] Charlie Chen, Sebastian Borgeaud, Geoffrey Irving, Jean-Baptiste Lespiau, Laurent Sifre, and John Jumper. 2023. Accelerating large language model decoding with speculative sampling. arXiv:2302.01318. Retrieved from <https://arxiv.org/abs/2302.01318>. (2023).
- [55] Lequn Chen, Zihao Ye, Yongji Wu, Danyang Zhuo, Luis Ceze, and Arvind Krishnamurthy. 2023. Punica: Multi-tenant LoRA serving. arXiv:2310.18547. Retrieved from <https://arxiv.org/abs/2310.18547>. (2023).
- [56] Lingjiao Chen, Matei Zaharia, and James Zou. 2023. FrugalGPT: How to use large language models while reducing cost and improving performance. arXiv:2305.05176. Retrieved from <https://arxiv.org/abs/2305.05176>. (2023).
- [57] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebggen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. 2021. Evaluating large language models trained on code. arXiv:2107.03374. Retrieved from <https://arxiv.org/abs/2107.03374>. (2021).
- [58] Shiyang Chen, Shaoyi Huang, Santosh Pandey, Bingbing Li, Guang R. Gao, Long Zheng, Caiwen Ding, and Hang Liu. 2021. Et: Re-thinking self-attention for transformer models on gpus. In *Proceedings of the HPCA 2021*. 1–18.
- [59] Shouyuan Chen, Sherman Wong, Liangjian Chen, and Yuandong Tian. 2023. Extending context window of large language models via positional interpolation. arXiv:2306.15595. Retrieved from <https://arxiv.org/abs/2306.15595>. (2023).
- [60] Wenhui Chen, Xueguang Ma, Xinyi Wang, and William W. Cohen. 2022. Program of thoughts prompting: Disentangling computation from reasoning for numerical reasoning tasks. arXiv:2211.12588. Retrieved from <https://arxiv.org/abs/2211.12588>. (2022).
- [61] Zhuoming Chen, Avner May, Ruslan Svirschevski, Yu-Hsun Huang, Max Ryabinin, Zhihao Jia, and Beidi Chen. 2024. Sequoia: Scalable and robust speculative decoding. *Proceedings of NeurIPS (2024)*, 129531–129563.
- [62] Zhuoming Chen, Ranajoy Sadhukhan, Zihao Ye, Yang Zhou, Jianyu Zhang, Niklas Nolte, Yuandong Tian, Matthijs Douze, Leon Bottou, Zhihao Jia, and Beidi Chen. 2024. Magicpig: Lsh sampling for efficient llm generation. In *Proceedings of the ICLR 2024*.
- [63] Alexis Chevalier, Alexander Wettig, Anirudh Ajith, and Danqi Chen. 2023. Adapting language models to compress contexts. arXiv:2305.14788. Retrieved from <https://arxiv.org/abs/2305.14788> (2023).
- [64] Wei-Lin Chiang, Zhuohan Li, Zi Lin, Ying Sheng, Zhanghao Wu, Hao Zhang, Lianmin Zheng, Siyuan Zhuang, Yonghao Zhuang, Joseph E. Gonzalez, Ion Stoica, and Eric P. Xing. 2023. Vicuna: An Open-Source Chatbot Impressing GPT-4 with 90%* ChatGPT Quality. (2023). Retrieved from <https://lmsys.org/blog/2023-03-30-vicuna/>
- [65] Rewon Child, Scott Gray, Alec Radford, and Ilya Sutskever. 2019. Generating long sequences with sparse transformers. arXiv:1904.10509. Retrieved from <https://arxiv.org/abs/1904.10509> (2019).
- [66] Krishna Teja Chitty-Venkata, Siddhisanket Raskar, Bharat Kale, Farah Ferdaus, Aditya Tanikanti, Ken Raffanetti, Valerie Taylor, Murali Emami, and Venkatram Vishwanath. 2024. LLM-inference-bench: Inference benchmarking of large language models on AI accelerators. In *Proceedings of the Workshops of SC 2024*. 1362–1379.
- [67] Jaewan Choi, Hailong Li, Byeongho Kim, Seunghwan Hwang, and Jung Ho Ahn. 2022. Accelerating transformer networks through recomposing softmax layers. In *Proceedings of the IISWC 2022*. 92–103.

- [68] Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, Maarten Bosma, Gaurav Mishra, Adam Roberts, Paul Barham, Hyung Won Chung, Charles Sutton, Sebastian Gehrmann, Parker Schuh, Kensen Shi, Sasha Tsvyashchenko, Joshua Maynez, Abhishek Rao, Parker Barnes, Yi Tay, Noam Shazeer, Vinodkumar Prabhakaran, Emily Reif, Nan Du, Ben Hutchinson, Reiner Pope, James Bradbury, Jacob Austin, Michael Isard, Guy Gur-Ari, Pengcheng Yin, Toju Duke, Anselm Levskaya, Sanjay Ghemawat, Sunipa Dev, Henryk Michalewski, Xavier Garcia, Vedant Misra, Kevin Robinson, Liam Fedus, Denny Zhou, Daphne Ippolito, David Luan, Hyeontaek Lim, Barret Zoph, Alexander Spiridonov, Ryan Sepassi, David Dohan, Shivani Agrawal, Mark Omernick, Andrew M. Dai, Thanumalayan Sankaranarayanan Pillai, Marie Pellat, Aitor Lewkowycz, Erica Moreira, Rewon Child, Oleksandr Polozov, Katherine Lee, Zongwei Zhou, Xuezhi Wang, Brennan Saeta, Mark Diaz, Orhan Firat, Michele Catasta, Jason Wei, Kathy Meier-Hellstern, Douglas Eck, Jeff Dean, Slav Petrov, and Noah Fiedel. 2022. Palm: Scaling language modeling with pathways. arXiv:2204.02311. Retrieved from <https://arxiv.org/abs/2204.02311> (2022).
- [69] Jacob K. Christopher, Brian R. Bartoldson, Tal Ben-Nun, Michael Cardei, Bhavya Kailkhura, and Ferdinando Fioretto. 2024. Speculative diffusion decoding: Accelerating language generation through diffusion. arXiv:2408.05636. Retrieved from <https://arxiv.org/abs/2408.05636> (2024).
- [70] Gonalo M. Correia, Vlad Niculae, and Andr  FT Martins. 2019. Adaptively sparse transformers. In *Proceedings of the EMNLP-IJCNLP 2019*.
- [71] R bert Csord s, Piotr Pi kos, and Kazuki Irie. 2023. SwitchHead: Accelerating transformers with mixture-of-experts attention. arXiv:2312.07987. Retrieved from <https://arxiv.org/abs/2312.07987>. (2023).
- [72] Fahim Dalvi, Maram Hasanain, Sabri Boughorbel, Basel Mousi, Samir Abdaljalil, Nizi Nazar, Ahmed Abdelali, Shammur Absar Chowdhury, Hamdy Mubarak, Ahmed Ali, Majd Hawasly, Nadir Durrani, and Firoj Alam. 2023. LLMebench: A flexible framework for accelerating LLMs benchmarking. arXiv:2308.04945. Retrieved from <https://arxiv.org/abs/2308.04945>. (2023).
- [73] Databricks. 2023. LLM Inference Performance Engineering: Best Practices. (2023). Retrieved November 25, 2023 from <https://www.databricks.com/blog/llm-inference-performance-engineering-best-practices>
- [74] Yann N. Dauphin, Angela Fan, Michael Auli, and David Grangier. 2017. Language modeling with gated convolutional networks. In *Proceedings of the ICML*. 933–941.
- [75] DeciAI. 2023. DeciLM 6B. (2023). Retrieved from <https://huggingface.co/Deci/DeciLM-6b>
- [76] Luciano Del Corro, Allie Del Giorno, Sahaj Agarwal, Bin Yu, Ahmed Awadallah, and Subhabrata Mukherjee. 2023. SkipDecode: Autoregressive Skip Decoding with Batching and Caching for Efficient LLM Inference. arXiv:2307.02628. Retrieved from <https://arxiv.org/abs/2307.02628>. (2023).
- [77] Tim Dettmers, Mike Lewis, Younes Belkada, and Luke Zettlemoyer. 2022. LLM.int8(): 8-bit matrix multiplication for transformers at scale. arXiv:2208.07339. Retrieved from <https://arxiv.org/abs/2208.07339>. (2022).
- [78] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. 2023. Qlora: Efficient finetuning of quantized llms. arXiv:2305.14314. Retrieved from <https://arxiv.org/abs/2305.14314>. (2023).
- [79] Tim Dettmers, Ruslan Svirschevski, Vage Egiazarian, Denis Kuznedelev, Elias Frantar, Saleh Ashkboos, Alexander Borzunov, Torsten Hoefer, and Dan Alistarh. 2023. SpQR: A sparse-quantized representation for near-lossless LLM weight compression. arXiv:2306.03078. Retrieved from <https://arxiv.org/abs/2306.03078>. (2023).
- [80] Tim Dettmers and Luke Zettlemoyer. 2023. The case for 4-bit precision: K-bit inference scaling laws. In *Proceedings of the ICLR 2023*.
- [81] Nolan Dey, Gurpreet Gosal, Hemant Khachane, William Marshall, Ribhu Pathria, Marvin Tom, Joel Hestness, and Zhiming (Charles) Chen. 2023. Cerebras-GPT: Open compute-optimal language models trained on the Cerebras wafer-scale cluster. arXiv:2304.03208. Retrieved from <https://arxiv.org/abs/2304.03208>. (2023).
- [82] Jiayu Ding, Shuming Ma, Li Dong, Xingxing Zhang, Shaohan Huang, Wenhui Wang, and Furu Wei. 2023. Longnet: Scaling transformers to 1,000,000,000 tokens. arXiv:2307.02486. Retrieved from <https://arxiv.org/abs/2307.02486>. (2023).
- [83] Juechu Dong, Boyuan Feng, Driss Guessous, Yanbo Liang, and Horace He. 2024. Flex attention: A programming model for generating optimized attention kernels. arXiv:2412.05496. Retrieved from <https://arxiv.org/abs/2412.05496>. (2024).
- [84] Xin Luna Dong, Seungwhan Moon, Yifan Ethan Xu, Kshitiz Malik, and Zhou Yu. 2023. Towards next-generation intelligent assistants leveraging LLM techniques. In *Proceedings of the KDD*. 5792–5793.
- [85] Yixin Dong, Charlie F. Ruan, Yaxing Cai, Ruihang Lai, Ziyi Xu, Yilong Zhao, and Tianqi Chen. 2024. Xgrammar: Flexible and efficient structured generation engine for large language models. arXiv:2411.15100. Retrieved from <https://arxiv.org/abs/2411.15100>. (2024).
- [86] Jiangsu Du, Jiazhi Jiang, Jiang Zheng, Hongbin Zhang, Dan Huang, and Yutong Lu. 2023. Improving computation and memory efficiency for real-world transformer inference on GPUs. *ACM TACO* (2023), 1–22.
- [87] Nan Du, Yanping Huang, Andrew M. Dai, Simon Tong, Dmitry Lepikhin, Yuanzhong Xu, Maxim Krikun, Yanqi Zhou, Adams Wei Yu, Orhan Firat, Barret Zoph, Liam Fedus, Maarten Bosma, Zongwei Zhou, Tao Wang, Yu Emma Wang, Kellie Webster, Marie Pellat, Kevin Robinson, Kathleen Meier-Hellstern, Toju Duke, Lucas Dixon, Kun Zhang,

- Quoc V Le, Yonghui Wu, Zhipeng Chen, and Claire Cui. 2022. Glam: Efficient scaling of language models with mixture-of-experts. In *Proceedings of the ICML 2022*. 5547–5569.
- [88] Murali Emani, Sam Foreman, Varuni Sastry, Zhen Xie, Siddhisanket Raskar, William Arnold, Rajeev Thakur, Venkatram Vishwanath, and Michael E. Papka. 2023. A comprehensive performance study of large language models on novel AI accelerators. arXiv:2310.04607. Retrieved from <https://arxiv.org/abs/2310.04607>. (2023).
- [89] Ahmad Faiz, Sotaro Kaneda, Ruhan Wang, Rita Osi, Parteek Sharma, Fan Chen, and Lei Jiang. 2023. LLMCarbon: Modeling the end-to-end carbon footprint of large language models. arXiv:2309.14393. Retrieved from <https://arxiv.org/abs/2309.14393>. (2023).
- [90] Angela Fan, Edouard Grave, and Armand Joulin. 2019. Reducing transformer depth on demand with structured dropout. In *Proceedings of the ICLR 2019*.
- [91] Jiarui Fang, Yang Yu, Chengduo Zhao, and Jie Zhou. 2021. Turbotransformers: An efficient gpu serving system for transformer models. In *Proceedings of the PPOPP 2021*. 389–402.
- [92] William Fedus, Barret Zoph, and Noam Shazeer. 2022. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *The Journal of Machine Learning Research* (2022), 5232–5270.
- [93] Pratik Fegade, Tianqi Chen, Phillip Gibbons, and Todd Mowry. 2022. The CoRa tensor compiler: Compilation for ragged tensors with minimal padding. *Proceedings of Machine Learning and Systems* (2022), 721–747.
- [94] Weizhi Fei, Xueyan Niu, Pingyi Zhou, Lu Hou, Bo Bai, Lei Deng, and Wei Han. 2023. Extending context window of large language models via semantic compression. arXiv:2312.09571. Retrieved from <https://arxiv.org/abs/2312.09571>. (2023).
- [95] Siyuan Feng, Bohan Hou, Hongyi Jin, Wuwei Lin, Junru Shao, Ruihang Lai, Zihao Ye, Lianmin Zheng, Cody Hao Yu, Yong Yu, and Tianqi Chen. 2023. Tensorir: An abstraction for automatic tensorized program optimization. In *Proceedings of the ASPLOS 2023*. 804–817.
- [96] Elias Frantar and Dan Alistarh. 2023. SparseGPT: Massive language models can be accurately pruned in one-shot. In *Proceedings of the ICML 2023*. 10323–10337.
- [97] Elias Frantar, Saleh Ashkboos, Torsten Hoefer, and Dan Alistarh. 2022. Gptq: Accurate post-training quantization for generative pre-trained transformers. arXiv:2210.17323. Retrieved from <https://arxiv.org/abs/2210.17323> (2022).
- [98] Elias Frantar, Saleh Ashkboos, Torsten Hoefer, and Dan Alistarh. 2022. OPTQ: Accurate quantization for generative pre-trained transformers. In *Proceedings of the ICLR 2022*.
- [99] Roy Frostig, Matthew James Johnson, and Chris Leary. 2018. Compiling machine learning programs via high-level tracing. *Systems for Machine Learning* (2018).
- [100] Daniel Y. Fu, Tri Dao, Khaled Kamal Saab, Armin W. Thomas, Atri Rudra, and Christopher Re. 2022. Hungry hungry hippos: Towards language modeling with state space models. In *Proceedings of the ICLR 2022*.
- [101] Yichao Fu, Peter Bailis, Ion Stoica, and Hao Zhang. 2024. Break the sequential dependency of LLM inference using LOOKAHEAD DECODING. In *Proceedings of the ICML 2024*. 14060–14079.
- [102] Yichao Fu, Junda Chen, Siqi Zhu, Zheyu Fu, Zhongdongming Dai, Aurick Qiao, and Hao Zhang. 2024. Efficiently serving LLM reasoning programs with certaintindex. arXiv:2412.20993. Retrieved from <https://arxiv.org/abs/2412.20993>. (2024).
- [103] Yao Fu, Leyang Xue, Yeqi Huang, Andrei-Octavian Brabete, Dmitrii Ustiugov, Yuvraj Patel, and Luo Mai. 2024. ServerlessLLM: Low-latency serverless inference for large language models. In *Proceedings of the OSDI 2024*. 135–153.
- [104] Trevor Gale, Deepak Narayanan, Cliff Young, and Matei Zaharia. 2023. MegaBlocks: Efficient sparse training with mixture-of-experts. *Proceedings of Machine Learning and Systems* (2023).
- [105] Bin Gao, Zhuomin He, Puru Sharma, Qingxuan Kang, Djordje Jevdjic, Junbo Deng, Xingkun Yang, Zhou Yu, and Pengfei Zuo. 2024. Cost-efficient large language model serving for multi-turn conversations with cached attention. In *Proceedings of the USENIX ATC 2024*. 111–126.
- [106] Shiwei Gao, Youmin Chen, and Jiwu Shu. 2025. Fast state restoration in LLM serving with hcache. In *Proceedings of the EuroSys 2025*.
- [107] Suyu Ge, Yunan Zhang, Liyuan Liu, Minjia Zhang, Jiawei Han, and Jianfeng Gao. 2023. Model tells you what to discard: Adaptive KV cache compression for LLMs. In *Proceedings of the WANTS@ NeurIPS 2023*.
- [108] Tao Ge, Jing Hu, Xun Wang, Si-Qing Chen, and Furu Wei. 2023. In-context autoencoder for context compression in a large language model. arXiv:2307.06945. Retrieved from <https://arxiv.org/abs/2307.06945>. (2023).
- [109] Tao Ge, Heming Xia, Xin Sun, Si-Qing Chen, and Furu Wei. 2022. Lossless acceleration for Seq2seq generation with aggressive decoding. arXiv:2205.10350. Retrieved from <https://arxiv.org/abs/2205.10350>. (2022).
- [110] Marjan Ghazvininejad, Omer Levy, Yinhan Liu, and Luke Zettlemoyer. 2019. Mask-predict: Parallel decoding of conditional masked language models. In *Proceedings of the EMNLP-IJCNLP 2019*. 6112–6121.
- [111] Marjan Ghazvininejad, Omer Levy, and Luke Zettlemoyer. 2020. Semi-autoregressive training improves mask-predict decoding. arXiv:2001.08785. Retrieved from <https://arxiv.org/abs/2001.08785>. (2020).

- [112] Amir Gholami, Sehoon Kim, Zhen Dong, Zhewei Yao, Michael W. Mahoney, and Kurt Keutzer. 2022. A survey of quantization methods for efficient neural network inference. In *Proceedings of the Low-Power Computer Vision*. 291–326.
- [113] In Gim, Guojun Chen, Seung-seob Lee, Nikhil Sarda, Anurag Khandelwal, and Lin Zhong. 2023. Prompt cache: Modular attention reuse for low-latency inference. arXiv:2311.04934. Retrieved from <https://arxiv.org/abs/2311.04934>. (2023).
- [114] Saurabh Goyal, Anamitra Roy Choudhury, Saurabh Raje, Venkatesan Chakaravarthy, Yogish Sabharwal, and Ashish Verma. 2020. PoWER-BERT: Accelerating BERT inference via progressive word-vector elimination. In *Proceedings of the ICML 2020*.
- [115] Albert Gu and Tri Dao. 2023. Mamba: Linear-time sequence modeling with selective state spaces. In *Proceedings of the COLM 2024*.
- [116] Albert Gu, Karan Goel, and Christopher Re. 2021. Efficiently modeling long sequences with structured state spaces. In *Proceedings of the ICLR 2021*.
- [117] J Gu, J Bradbury, C Xiong, VOK Li, and R Socher. 2018. Non-autoregressive neural machine translation. In *Proceedings of the ICLR 2018*.
- [118] Jiatao Gu and Xiang Kong. 2021. Fully Non-autoregressive neural machine translation: Tricks of the trade. In *Proceedings of the ACL Findings*. 120–133.
- [119] Yuxian Gu, Li Dong, Furu Wei, and Minlie Huang. 2024. MiniLLM: Knowledge distillation of large language models. In *Proceedings of the ICLR 2024*.
- [120] Jashwant Raj Gunasekaran, Cyan Subhra Mishra, Prashanth Thinakaran, Bikash Sharma, Mahmut Taylan Kandemir, and Chita R Das. 2022. Cocktail: A multidimensional optimization for model serving in cloud. In *Proceedings of the NSDI 2022*. 1041–1057.
- [121] Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, 2025. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. arXiv:2501.12948. Retrieved from <https://arxiv.org/abs/2501.12948>. (2025).
- [122] Junliang Guo, Xu Tan, Di He, Tao Qin, Linli Xu, and Tie-Yan Liu. 2019. Non-autoregressive neural machine translation with enhanced decoder input. In *Proceedings of the AAAI*. 3723–3730.
- [123] Liwei Guo, Wonkyo Choe, and Felix Xiaozhu Lin. 2023. STI: Turbocharge NLP inference at the edge via elastic pipelining. In *Proceedings of the ASPLOS*. 791–803.
- [124] Qipeng Guo, Xipeng Qiu, Pengfei Liu, Yunfan Shao, Xiangyang Xue, and Zheng Zhang. 2019. Star-transformer. In *Proceedings of the NAACL-HLT*. 1315–1325.
- [125] Ankit Gupta and Jonathan Berant. 2020. Gmat: Global memory augmentation for transformers. arXiv:2006.03274. Retrieved from <https://arxiv.org/abs/2006.03274>. (2020).
- [126] Ankit Gupta, Guy Dar, Shaya Goodman, David Ciprut, and Jonathan Berant. 2021. Memory-efficient transformers via top-k attention. In *Proceedings of the 2nd Workshop on Simple and Efficient Natural Language Processing*. 39–52.
- [127] Manish Gupta and Puneet Agrawal. 2022. Compression of deep learning models for text: A survey. *ACM Transactions on Knowledge Discovery from Data (TKDD)* (2022), 1–55.
- [128] Mingcong Han, Hanze Zhang, Rong Chen, and Haibo Chen. 2022. Microsecond-scale preemption for concurrent GPU-accelerated DNN inferences. In *Proceedings of the OSDI 2022*. 539–558.
- [129] Bobby He and Thomas Hofmann. 2023. Simplifying transformer blocks. arXiv:2311.01906. Retrieved from <https://arxiv.org/abs/2311.01906>. (2023).
- [130] Jiaao He, Jidong Zhai, Tiago Antunes, Haojie Wang, Fuwen Luo, Shangfeng Shi, and Qin Li. 2022. FasterMoE: Modeling and optimizing training of large-scale dynamic pre-trained models. In *Proceedings of the PPOPP 2022*. 120–134.
- [131] Xuanli He, Iman Keivanloo, Yi Xu, Xiang He, Belinda Zeng, Santosh Rajagopalan, and Trishul Chilimbi. 2021. Magic pyramid: Accelerating inference with early exiting and token pruning. arXiv:2111.00230. Retrieved from <https://arxiv.org/abs/2111.00230>. (2021).
- [132] Zhenyu He, Zexuan Zhong, Tianle Cai, Jason D. Lee, and Di He. 2023. REST: Retrieval-based speculative decoding. arXiv:2311.08252. Retrieved from <https://arxiv.org/abs/2311.08252>. (2023).
- [133] Ke Hong, Guohao Dai, Jiaming Xu, Qiuli Mao, Xiuhong Li, Jun Liu, Kangdi Chen, Hanyu Dong, and Yu Wang. 2023. FlashDecoding++: Faster large language model inference on GPUs. arXiv:2311.01282. Retrieved from <https://arxiv.org/abs/2311.01282>. (2023).
- [134] Coleman Hooper, Sehoon Kim, Hiva Mohammadzadeh, Hasan Genc, Kurt Keutzer, Amir Gholami, and Sophia Shao. 2023. SPEED: Speculative pipelined execution for efficient decoding. arXiv:2310.12072. Retrieved from <https://arxiv.org/abs/2310.12072>. (2023).
- [135] Cunchen Hu, Heyang Huang, Liangliang Xu, Xusheng Chen, Jiang Xu, Shuang Chen, Hao Feng, Chenxi Wang, Sa Wang, Yungang Bao, Ninghui Sun, and Yizhou Shan. 2024. Inference without interference: Disaggregate llm inference for mixed downstream workloads. arXiv:2401.11181. Retrieved from <https://arxiv.org/abs/2401.11181>. (2024).
- [136] Haiyang Huang, Newsha Ardalani, Anna Sun, Liu Ke, Hsien-Hsin S. Lee, Anjali Sridhar, Shruti Bhosale, Carole-Jean Wu, and Benjamin Lee. 2023. Towards MoE deployment: Mitigating inefficiencies in mixture-of-expert (MoE) inference. arXiv:2303.06182. Retrieved from <https://arxiv.org/abs/2303.06182>. (2023).

- [137] Kaiyu Huang, Hao Wu, Zubo Shi, Han Zou, Minchen Yu, and Qingjiang Shi. 2025. SpecServe: Efficient and SLO-aware large language model serving with adaptive speculative decoding. arXiv:2503.05096. Retrieved from <https://arxiv.org/abs/2503.05096>. (2025).
- [138] Changho Hwang, Wei Cui, Yifan Xiong, Ziyue Yang, Ze Liu, Han Hu, Zilong Wang, Rafael Salas, Jithin Jose, Prabhat Ram, Joe Chau, Peng Cheng, Fan Yang, Mao Yang, and Yongqiang Xiong. 2023. Tutel: Adaptive mixture-of-experts at scale. *Proceedings of Machine Learning and Systems* (2023).
- [139] Régis Pierrard Ilyas Moutawakil. 2023. LLM-Perf Leaderboard. Retrieved from <https://huggingface.co/spaces/optimum/llm-perf-leaderboard>. (2023).
- [140] Mikhail Isaev, Nic McDonald, Larry Dennison, and Richard Vuduc. 2023. Calculon: A methodology and tool for high-level co-design of systems and large language models. In *Proceedings of the SC 2023*. 1–14.
- [141] Berivan Isik, Hermann Kumbong, Wanyi Ning, Xiaozhe Yao, Sanmi Koyejo, and Ce Zhang. 2023. GPT-Zip: Deep compression of finetuned large language models. In *Proceedings of the Workshop on Efficient Systems for Foundation Models@ ICML 2023*.
- [142] Sam Ade Jacobs, Masahiro Tanaka, Chengming Zhang, Minjia Zhang, Shuaiwen Leon Song, Samyam Rajbhandari, and Yuxiong He. 2023. Deepspeed ulysses: System optimizations for enabling training of extreme long sequence transformer models. arXiv:2309.14509. Retrieved from <https://arxiv.org/abs/2309.14509>. (2023).
- [143] Ajay Jaiswal, Zhe Gan, Xianzhi Du, Bowen Zhang, Zhangyang Wang, and Yinfei Yang. 2023. Compressing LLMs: The truth is rarely pure and never simple. arXiv:2310.01382. Retrieved from <https://arxiv.org/abs/2310.01382>. (2023).
- [144] Doohyuk Jang, Sihwan Park, June Yong Yang, Yeonsung Jung, Jihun Yun, Souvik Kundu, Sung-Yub Kim, and Eunho Yang. 2024. LANTERN: Accelerating visual autoregressive models with relaxed speculative decoding. In *Proceedings of the ICLR 2024*.
- [145] Byungsoo Jeon, Mengdi Wu, Shiyi Cao, Sunghyun Kim, Sunghyun Park, Neeraj Aggarwal, Colin Unger, Daiyaan Arfeen, Peiyuan Liao, Xupeng Miao, Mohammad Alizadeh, Gregory R. Ganger, Tianqi Chen, and Zhihao Jia. 2025. GraphPipe: Improving performance and scalability of DNN training with graph pipeline parallelism. In *Proceedings of the ASPLOS 2025*. 557–571.
- [146] Zhihao Jia, Oded Padon, James Thomas, Todd Warszawski, Matei Zaharia, and Alex Aiken. 2019. TASO: Optimizing deep learning computation with automatic generation of graph substitutions. In *Proceedings of the SOSP 2019*. 47–62.
- [147] Zhihao Jia, Matei Zaharia, and Alex Aiken. 2019. Beyond data and model parallelism for deep neural networks. *Proceedings of Machine Learning and Systems* (2019), 1–13.
- [148] Albert Q. Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, Léo Renard Lavaud, Marie-Anne Lachaux, Pierre Stock, Teven Le Scao, Thibaut Lavril, Thomas Wang, Timothée Lacroix, and William El Sayed. 2023. Mistral 7B. arXiv:2310.06825. Retrieved from <https://arxiv.org/abs/2310.06825>. (2023).
- [149] Huiqiang Jiang, Qianhui Wu, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. 2023. LlmLingua: Compressing prompts for accelerated inference of large language models. arXiv:2310.05736. Retrieved from <https://arxiv.org/abs/2310.05736>. (2023).
- [150] Huiqiang Jiang, Qianhui Wu, Xufang Luo, Dongsheng Li, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. 2024. LongLLM-Lingua: Accelerating and enhancing LLMs in long context scenarios via prompt compression. In *Proceedings of the ACL 2024*.
- [151] Youhe Jiang, Fangcheng Fu, Xiaozhe Yao, Guoliang He, Xupeng Miao, Ana Klimovic, Bin Cui, Binhang Yuan, and Eiko Yoneki. 2025. Demystifying cost-efficiency in LLM serving over heterogeneous GPUs. arXiv:2502.00722. Retrieved from <https://arxiv.org/abs/2502.00722>. (2025).
- [152] Youhe Jiang, Ran Yan, Xiaozhe Yao, Beidi Chen, and Binhang Yuan. 2023. HexGen: Generative inference of foundation model over heterogeneous decentralized environment. arXiv:2311.11514. Retrieved from <https://arxiv.org/abs/2311.11514>. (2023).
- [153] Xiaoqi Jiao, Yichun Yin, Lifeng Shang, Xin Jiang, Xiao Chen, Linlin Li, Fang Wang, and Qun Liu. 2020. TinyBERT: Distilling BERT for natural language understanding. In *Proceedings of the EMNLP Findings*. 4163–4174.
- [154] Hongyi Jin, Ruihang Lai, Charlie F. Ruan, Yingcheng Wang, Todd C. Mowry, Xupeng Miao, Zhihao Jia, and Tianqi Chen. 2024. A system for microservicing of LLMs. arXiv:2412.12488. Retrieved from <https://arxiv.org/abs/2412.12488>. (2024).
- [155] Yunho Jin, Chun-Feng Wu, David Brooks, and Gu-Yeon Wei. 2023. S³: Increasing GPU utilization during generative inference for higher throughput. arXiv:2306.06000. Retrieved from <https://arxiv.org/abs/2306.06000>. (2023).
- [156] A Jo. 2023. The promise and peril of generative AI. *Nature* (2023), 214–216.
- [157] Norm Jouppi, George Kurian, Sheng Li, Peter Ma, Rahul Nagarajan, Lifeng Nai, Nishant Patil, Suvinay Subramanian, Andy Swing, Brian Towles, Cliff Young, Xiang Zhou, Zongwei Zhou, and David Patterson. 2023. Tpu v4: An optically reconfigurable supercomputer for machine learning with hardware support for embeddings. In *Proceedings of the ISCA 2023*. 1–14.

- [158] Jungo Kasai, Nikolaos Pappas, Hao Peng, James Cross, and Noah Smith. 2020. Deep encoder, shallow decoder: Reevaluating non-autoregressive machine translation. In *Proceedings of the ICLR 2020*.
- [159] Navdeep Katel, Vivek Khandelwal, and Uday Bondhugula. 2022. MLIR-based code generation for GPU tensor cores. In *Proceedings of the 31st ACM SIGPLAN International Conference on Compiler Construction*. 117–128.
- [160] Angelos Katharopoulos, Apoorv Vyas, Nikolaos Pappas, and François Fleuret. 2020. Transformers are rnns: Fast autoregressive transformers with linear attention. In *Proceedings of the ICML 2020*. 5156–5165.
- [161] Urvashi Khandelwal, He He, Peng Qi, and Dan Jurafsky. 2018. Sharp nearby, fuzzy far away: How neural language models use context. In *Proceedings of the ACL 2018*. 284–294.
- [162] Jeonghoon Kim, Jung Hyun Lee, Sungdong Kim, Joonsuk Park, Kang Min Yoo, Se Jung Kwon, and Dongsoo Lee. 2023. Memory-efficient fine-tuning of compressed large language models via sub-4-bit integer quantization. arXiv:2305.14152. Retrieved from <https://arxiv.org/abs/2305.14152>. (2023).
- [163] Sehoon Kim, Coleman Hooper, Amir Gholami, Zhen Dong, Xiuyu Li, Sheng Shen, Michael W. Mahoney, and Kurt Keutzer. 2023. SqueezeLLM: Dense-and-sparse quantization. arXiv:2306.07629. Retrieved from <https://arxiv.org/abs/2306.07629>. (2023).
- [164] Sehoon Kim, Coleman Hooper, Thanakul Wattanawong, Minwoo Kang, Ruohan Yan, Hasan Genc, Grace Dinh, Qijing Huang, Kurt Keutzer, Michael W. Mahoney, Yakun Sophia Shao, and Amir Gholami. 2023. Full stack optimization of transformer inference: a survey. arXiv:2302.14017. Retrieved from <https://arxiv.org/abs/2302.14017>. (2023).
- [165] Sehoon Kim, Karttikeya Mangalam, Jitendra Malik, Michael W. Mahoney, Amir Gholami, and Kurt Keutzer. 2023. Big little transformer decoder. arXiv:2302.07863. Retrieved from <https://arxiv.org/abs/2302.07863>. (2023).
- [166] Nikita Kitaev, Lukasz Kaiser, and Anselm Levskaya. 2019. Reformer: The efficient transformer. In *Proceedings of the ICLR 2019*.
- [167] Jun Kong, Jin Wang, Liang-Chih Yu, and Xuejie Zhang. 2022. Accelerating inference for pretrained language models by unified multi-perspective early exiting. In *Proceedings of the COLING*. 4677–4686.
- [168] Vijay Anand Korthikanti, Jared Casper, Sangkug Lym, Lawrence McAfee, Michael Andersch, Mohammad Shoebybi, and Bryan Catanzaro. 2023. Reducing activation recomputation in large transformer models. *Proceedings of Machine Learning and Systems* (2023), 341–353.
- [169] Sneha Kudugunta, Yanping Huang, Ankur Bapna, Maxim Krikun, Dmitry Lepikhin, Minh-Thang Luong, and Orhan Firat. 2021. Beyond distillation: Task-level mixture-of-experts for efficient inference. In *Proceedings of the EMNLP Findings*. 3577–3599.
- [170] Eldar Kurtic, Elias Frantar, and Dan Alistarh. 2023. Ziplm: Hardware-aware structured pruning of language models. arXiv:2302.04089. Retrieved from <https://arxiv.org/abs/2302.04089>. (2023).
- [171] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the SOSP 2023*. 611–626.
- [172] Ruihang Lai, Junru Shao, Siyuan Feng, Steven S. Lyubomirsky, Bohan Hou, Wuwei Lin, Zihao Ye, Hongyi Jin, Yuchen Jin, Jiawei Liu, Lesheng Jin, Yaxing Cai, Ziheng Jiang, Yong Wu, Sunghyun Park, Prakalp Srivastava, Jared G. Roesch, Todd C. Mowry, and Tianqi Chen. 2025. Relax: Composable abstractions for end-to-end dynamic machine learning. In *Proceedings of the ASPLOS 2025*.
- [173] Jason Lee, Elman Mansimov, and Kyunghyun Cho. 2018. Deterministic non-autoregressive neural sequence modeling by iterative refinement. In *Proceedings of the EMNLP 2018*.
- [174] Wonbeom Lee, Jungi Lee, Junghwan Seo, and Jaewoong Sim. 2024. InfiniGen: Efficient generative inference of large language models with dynamic KV cache management. In *Proceedings of the OSDI 2024*. 155–172.
- [175] Benjamin Lefaudeux, Francisco Massa, Diana Liskovich, Wenhan Xiong, Vittorio Caggiano, Sean Naren, Min Xu, Jieru Hu, Marta Tintore, Susan Zhang, et al. 2022. xFormers: A modular and hackable Transformer modelling library. Retrieved November 25, 2023 from <https://github.com/facebookresearch/xformers>. (2022). Commit: fb349a.
- [176] Dmitry Lepikhin, HyoukJoong Lee, Yuanzhong Xu, Dehao Chen, Orhan Firat, Yanping Huang, Maxim Krikun, Noam Shazeer, and Zhifeng Chen. 2020. GShard: Scaling giant models with conditional computation and automatic sharding. In *Proceedings of the ICLR 2020*.
- [177] Yaniv Leviathan, Matan Kalman, and Yossi Matias. 2023. Fast inference from transformers via speculative decoding. In *Proceedings of the ICML 2023*. 19274–19286.
- [178] Jiamin Li, Yimin Jiang, Yibo Zhu, Cong Wang, and Hong Xu. 2023. Accelerating distributed MoE training and inference with Lina. In *Proceedings of the USENIX ATC 2023*. 945–959.
- [179] Lei Li, Yankai Lin, Deli Chen, Shuhuai Ren, Peng Li, Jie Zhou, and Xu Sun. 2020. Cascadebert: Accelerating inference of pre-trained language models via calibrated complete models cascade. arXiv:2012.14682. Retrieved from <https://arxiv.org/abs/2012.14682>. (2020).
- [180] Qingyuan Li, Ran Meng, Yiduo Li, Bo Zhang, Liang Li, Yifan Lu, Xiangxiang Chu, Yerui Sun, and Yuchen Xie. 2023. A speed odyssey for deployable quantization of LLMs. arXiv:2311.09550. Retrieved from <https://arxiv.org/abs/2311.09550>. (2023).

- [181] Yucheng Li, Bo Dong, Frank Guerin, and Chenghua Lin. 2023. Compressing context to enhance inference efficiency of large language models. In *Proceedings of the EMNLP 2023*. 6342–6353.
- [182] Yanyang Li, Ye Lin, Tong Xiao, and Jingbo Zhu. 2021. An efficient transformer decoder with compressed sub-layers. In *Proceedings of the AAAI 2021*. 13315–13323.
- [183] Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. 2024. EAGLE: speculative sampling requires rethinking feature uncertainty. In *Proceedings of the ICML 2024*. 28935–28948.
- [184] Yixiao Li, Yifan Yu, Qingru Zhang, Chen Liang, Pengcheng He, Weizhu Chen, and Tuo Zhao. 2023. LoSparse: Structured compression of large language models based on low-rank and sparse approximation. In *Proceedings of the ICML 2023*.
- [185] Zikun Li, Zhuofu Chen, Remi Delacourt, Gabriele Oliaro, Zeyu Wang, Qinghan Chen, Shuhuai Lin, April Yang, Zhihao Zhang, Zhuoming Chen, Sean Lai, Xinhao Cheng, Xupeng Miao, and Zhihao Jia. 2025. AdaServe: SLO-customized LLM serving with fine-grained speculative decoding. arXiv:2501.12162. Retrieved from <https://arxiv.org/abs/2501.12162>. (2025).
- [186] Zhuohan Li, Lianmin Zheng, Yinmin Zhong, Vincent Liu, Ying Sheng, Xin Jin, Yanping Huang, Zhifeng Chen, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. 2023. AlpaServe: Statistical multiplexing with model parallelism for deep learning serving. In *Proceedings of the OSDI 2023*. 663–679.
- [187] Kaiyuan Liao, Yi Zhang, Xuancheng Ren, Qi Su, Xu Sun, and Bin He. 2021. A global past-future early exit method for accelerating inference of pre-trained language models. In *Proceedings of the NAACL 2021*. 2013–2023.
- [188] Chaofan Lin, Zhenhua Han, Chengruidong Zhang, Yuqing Yang, Fan Yang, Chen Chen, and Lili Qiu. 2024. Parrot: Efficient serving of LLM-based applications with semantic variable. In *Proceedings of the OSDI 2024*. 929–945.
- [189] Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Wei-Ming Chen, Wei-Chen Wang, Guangxuan Xiao, Xingyu Dang, Chuang Gan, and Song Han. 2024. Awq: Activation-aware weight quantization for on-device llm compression and acceleration. *Proceedings of Machine Learning and Systems* 6 (2024), 87–100.
- [190] Aixin Liu, Bei Feng, Bin Wang, Bingxuan Wang, Bo Liu, Chenggang Zhao, Chengqi Deng, Chong Ruan, Damai Dai, Daya Guo, 2024. Deepseek-v2: A strong, economical, and efficient mixture-of-experts language model. arXiv:2405.04434. Retrieved from <https://arxiv.org/abs/2405.04434>. (2024).
- [191] Fangcheng Liu, Yehui Tang, Zhenhua Liu, Yunsheng Ni, Duyu Tang, Kai Han, and Yunhe Wang. 2024. Kangaroo: Lossless self-speculative decoding for accelerating LLMs via double early exiting. *Proceedings of NeurIPS* (2024), 11946–11965.
- [192] Hao Liu, Matei Zaharia, and Pieter Abbeel. 2023. Ring attention with blockwise transformers for near-infinite context. arXiv:2310.01889. Retrieved from <https://arxiv.org/abs/2310.01889>. (2023).
- [193] Jingyu Liu, Beidi Chen, and Ce Zhang. 2025. Speculative prefill: Turbocharging TTFT with lightweight and training-free token importance estimation. arXiv:2502.02789. Retrieved from <https://arxiv.org/abs/2502.02789>. (2025).
- [194] Jiachen Liu, Jae-Won Chung, Zhiyu Wu, Fan Lai, Myungjin Lee, and Mosharaf Chowdhury. 2024. Andes: Defining and enhancing quality-of-experience in llm-based text streaming services. arXiv:2404.16283. Retrieved from <https://arxiv.org/abs/2404.16283>. (2024).
- [195] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. 2023. Lost in the middle: How language models use long contexts. arXiv:2307.03172. Retrieved from <https://arxiv.org/abs/2307.03172>. (2023).
- [196] Weijie Liu, Peng Zhou, Zhiruo Wang, Zhe Zhao, Haotang Deng, and Qi Ju. 2020. FastBERT: A Self-distilling BERT with adaptive inference time. In *Proceedings of the ACL 2020*. 6035–6044.
- [197] Xiaoxuan Liu, Lanxiang Hu, Peter Bailis, Ion Stoica, Zhijie Deng, Alvin Cheung, and Hao Zhang. 2023. Online speculative decoding. arXiv:2310.07177. Retrieved from <https://arxiv.org/abs/2310.07177>. (2023).
- [198] Yuhai Liu, Hanchen Li, Kuntai Du, Jiayi Yao, Yihua Cheng, Yuyang Huang, Shan Lu, Michael Maire, Henry Hoffmann, Ari Holtzman, Siddhant Ray, Qizheng Zhang, Ganesh Ananthanarayanan, and Junchen Jiang. 2024. CacheGen: Fast context loading for language model applications. In *Proceedings of the SIGCOMM 2024*.
- [199] Zichang Liu, Aditya Desai, Fangshuo Liao, Weitao Wang, Victor Xie, Zhaozhuo Xu, Anastasios Kyrillidis, and Anshumali Shrivastava. 2023. Scissorhands: Exploiting the persistence of importance hypothesis for LLM KV cache compression at test time. arXiv:2305.17118. Retrieved from <https://arxiv.org/abs/2305.17118>. (2023).
- [200] Zechun Liu, Barlas Oguz, Changsheng Zhao, Ernie Chang, Pierre Stock, Yashar Mehdad, Yangyang Shi, Raghuraman Krishnamoorthi, and Vikas Chandra. 2023. LLM-QAT: Data-free quantization aware training for large language models. arXiv:2305.17888. Retrieved from <https://arxiv.org/abs/2305.17888>. (2023).
- [201] Zichang Liu, Yue Wang, Tri Dao, Tianyi Zhou, Binhang Yuan, Zhao Song, Anshumali Shrivastava, Ce Zhang, Yuandong Tian, Christopher Re, and Beidi Chen. 2023. Deja vu: Contextual sparsity for efficient llms at inference time. In *Proceedings of the ICML 2023*.
- [202] Wen-jie Lu, Zhicong Huang, Zhen Gu, Jingyu Li, Jian Liu, Kui Ren, Cheng Hong, Tao Wei, and WenGuang Chen. 2023. BumbleBee: Secure two-party inference framework for large transformers. *Cryptology ePrint Archive* (2023).

- [203] Xinyin Ma, Gongfan Fang, and Xinchao Wang. 2023. LLM-Pruner: On the structural pruning of large language models. arXiv:2305.11627. Retrieved from <https://arxiv.org/abs/2305.11627>. (2023).
- [204] Ziming Mao, Tian Xia, Zhanghao Wu, Wei-Lin Chiang, Tyler Griggs, Romil Bhardwaj, Zongheng Yang, Scott Shenker, and Ion Stoica. 2024. Skyserve: Serving ai models across regions and clouds with spot instances. arXiv:2411.01438. Retrieved from <https://arxiv.org/abs/2411.01438>. (2024).
- [205] Ruben Mayer and Hans-Arno Jacobsen. 2020. Scalable deep learning on distributed infrastructures: Challenges, techniques, and tools. *ACM Computing Surveys (CSUR)* (2020), 1–37.
- [206] Harsh Mehta, Ankit Gupta, Ashok Cutkosky, and Behnam Neyshabur. 2022. Long range language modeling via gated state spaces. In *Proceedings of the ICLR 2022*.
- [207] Yixuan Mei, Yonghao Zhuang, Xupeng Miao, Juncheng Yang, Zhihao Jia, and Rashmi Vinayak. 2025. Helix: Serving large language models over heterogeneous GPUs and network via max-flow. In *Proceedings of the ASPLOS 2025*. 586–602.
- [208] Zhiyu Mei, Wei Fu, Kaiwei Li, Guangju Wang, Huanchen Zhang, and Yi Wu. 2024. Realhf: Optimized rlhf training for large language models through parameter reallocation. arXiv:2406.14088. Retrieved from <https://arxiv.org/abs/2406.14088>. (2024).
- [209] Xupeng Miao, Zhihao Jia, and Bin Cui. 2024. Demystifying data management for large language models. In *Proceedings of the Companion of the 2024 International Conference on Management of Data*. 547–555.
- [210] Xupeng Miao, Gabriele Oliaro, Xinhao Cheng, Mengdi Wu, Colin Unger, and Zhihao Jia. 2024. FlexLLM: A system for co-serving large language model inference and parameter-efficient finetuning. arXiv:2402.18789. Retrieved from <https://arxiv.org/abs/2402.18789>. (2024).
- [211] Xupeng Miao, Gabriele Oliaro, Zhihao Zhang, Xinhao Cheng, Zeyu Wang, Zhengxin Zhang, Rae Ying Yee Wong, Alan Zhu, Lijie Yang, Xiaoxiang Shi, Chunan Shi, Zhuoming Chen, Daiyaan Arfeen, Reyna Abhyankar, and Zhihao Jia. 2024. Specinfer: Accelerating large language model serving with tree-based speculative inference and verification. In *Proceedings of the ASPLOS 2024*. 932–949.
- [212] Xupeng Miao, Chunan Shi, Jiangfei Duan, Xiaoli Xi, Dahua Lin, Bin Cui, and Zhihao Jia. 2024. SpotServe: Serving generative large language models on preemptible instances. In *Proceedings of the ASPLOS 2024*. 1112–1127.
- [213] Xupeng Miao, Yujie Wang, Youhe Jiang, Chunan Shi, Xiaonan Nie, Hailin Zhang, and Bin Cui. 2023. Galvatron: Efficient transformer training over multiple GPUs using automatic parallelism. *Proc. VLDB Endow.* (2023), 470–479.
- [214] Paul Michel, Omer Levy, and Graham Neubig. 2019. Are sixteen heads really better than one? *Proc. of NeurIPS* (2019).
- [215] Maxim Milakov and Natalia Gimelshein. 2018. Online normalizer calculation for softmax. arXiv:1805.02867. Retrieved from <https://arxiv.org/abs/1805.02867>. (2018).
- [216] Asit Mishra, Jorge Albericio Latorre, Jeff Pool, Darko Stosic, Dusan Stosic, Ganesh Venkatesh, Chong Yu, and Paulius Micikevicius. 2021. Accelerating sparse deep neural networks. arXiv:2104.08378. Retrieved from <https://arxiv.org/abs/2104.08378>. (2021).
- [217] Ali Modarressi, Hosein Mohebbi, and Mohammad Taher Pilehvar. 2022. Adapter: Speeding up inference by adaptive length reduction. arXiv:2203.08991. Retrieved from <https://arxiv.org/abs/2203.08991>. (2022).
- [218] Amirkeivan Mohtashami and Martin Jaggi. 2023. Landmark attention: Random-access infinite context length for transformers. arXiv:2305.16300. Retrieved from <https://arxiv.org/abs/2305.16300>. (2023).
- [219] Giovanni Monea, Armand Joulin, and Edouard Grave. 2023. PaSS: Parallel speculative sampling. arXiv:2311.13581. Retrieved from <https://arxiv.org/abs/2311.13581>. (2023).
- [220] Jesse Mu, Xiang Lisa Li, and Noah Goodman. 2023. Learning to compress prompts with gist tokens. In *Proceedings of the NeurIPS 2023*.
- [221] Dor Muhlgay, Ori Ram, Inbal Magar, Yoav Levine, Nir Ratner, Yonatan Belinkov, Omri Abend, Kevin Leyton-Brown, Amnon Shashua, and Yoav Shoham. 2023. Generating benchmarks for factuality evaluation of language models. arXiv:2307.06908. Retrieved from <https://arxiv.org/abs/2307.06908>. (2023).
- [222] Kabir Nagrecha and Arun Kumar. 2023. Saturn: An optimized data system for large model deep learning workloads. arXiv:2309.01226. Retrieved from <https://arxiv.org/abs/2309.01226>. (2023).
- [223] Deepak Narayanan, Amar Phanishayee, Kaiyu Shi, Xie Chen, and Matei Zaharia. 2021. Memory-efficient pipeline-parallel dnn training. In *Proceedings of the ICML 2021*. 7937–7947.
- [224] Deepak Narayanan, Keshav Santhanam, Peter Henderson, Rishi Bommasani, Tony Lee, and Percy Liang. 2023. Cheaply estimating inference efficiency metrics for autoregressive transformer models. In *Proceedings of the NeurIPS 2023*.
- [225] Kelvin KW Ng, Henri Maxime Demoulin, and Vincent Liu. 2023. Paella: Low-latency model serving with software-defined GPU scheduling. In *Proceedings of the SOSP 2023*. 595–610.
- [226] Xiaonan Nie, Xupeng Miao, Shijie Cao, Lingxiao Ma, Qibin Liu, Jilong Xue, Youshan Miao, Yi Liu, Zhi Yang, and Bin Cui. 2021. Evomoe: An evolutionary mixture-of-experts training framework via dense-to-sparse gate. arXiv:2112.14397. Retrieved from <https://arxiv.org/abs/2112.14397>. (2021).

- [227] Xiaonan Nie, Xupeng Miao, Zilong Wang, Zichao Yang, Jilong Xue, Lingxiao Ma, Gang Cao, and Bin Cui. 2023. FlexMoE: Scaling large-scale sparse pre-trained model training via dynamic device placement. *Proceedings of the ACM on Management of Data* (2023), 1–19.
- [228] Hyungjun Oh, Kihong Kim, Jaemin Kim, Sungkyun Kim, Junyeol Lee, Du-seong Chang, and Jiwon Seo. 2024. Exegpt: Constraint-aware resource scheduling for llm inference. In *Proceedings of the ASPLOS 2024*. 369–384.
- [229] Gabriele Oliaro, Zhihao Jia, Daniel Campos, and Aurick Qiao. 2024. SuffixDecoding: A model-free approach to speeding up large language model inference. arXiv:2411.04975. Retrieved from <https://arxiv.org/abs/2411.04975>. (2024).
- [230] Junier B. Oliva, Barnabás Póczos, and Jeff Schneider. 2017. The statistical recurrent unit. In *Proceedings of the ICML*. 2671–2680.
- [231] Antonio Orvieto, Samuel L. Smith, Albert Gu, Anushan Fernando, Caglar Gulcehre, Razvan Pascanu, and Soham De. 2023. Resurrecting recurrent neural networks for long sequences. arXiv:2303.06349. Retrieved from <https://arxiv.org/abs/2303.06349>. (2023).
- [232] Charles Packer, Vivian Fang, Shishir G. Patil, Kevin Lin, Sarah Wooders, and Joseph E. Gonzalez. 2023. MemGPT: Towards LLMs as operating systems. arXiv:2310.08560. Retrieved from <https://arxiv.org/abs/2310.08560>. (2023).
- [233] Matteo Pagliardini, Daniele Paliotta, Martin Jaggi, and François Fleuret. 2023. Faster causal attention over large sequences through sparse flash attention. arXiv:2306.01160. Retrieved from <https://arxiv.org/abs/2306.01160>. (2023).
- [234] Rui Pan, Zhuang Wang, Zhen Jia, Can Karakus, Luca Zancato, Tri Dao, Yida Wang, and Ravi Netravali. 2024. Marconi: Prefix caching for the era of hybrid llms. arXiv:2411.19379. Retrieved from <https://arxiv.org/abs/2411.19379>. (2024).
- [235] Gunho Park, Minsub Kim, Sungjae Lee, Jeonghoon Kim, Beomseok Kwon, Se Jung Kwon, Byeongwook Kim, Youngjoo Lee, Dongsoo Lee, and Baeseong Park. 2022. LUT-GEMM: Quantized matrix multiplication based on LUTs for efficient inference in large-scale generative language models. In *Proceedings of the ICLR 2022*.
- [236] Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Íñigo Goiri, Saeed Maleki, and Ricardo Bianchini. 2024. Splitwise: Efficient generative llm inference using phase splitting. In *Proceedings of the ISCA 2024*. 118–132.
- [237] Bo Peng, Eric Alcaide, Quentin Anthony, Alon Albalak, Samuel Arcadinho, Huanqi Cao, Xin Cheng, Michael Chung, Matteo Grella, Stella Biderman, Kranthi Kiran GV, Xuzheng He, Haowen Hou, Jiaju Lin, Przemyslaw Kazienko, Jan Kocon, Jiaming Kong, Bartłomiej Koptyra, Hayden Lau, Krishna Sri Ipsit Mantri, Ferdinand Mom, Atsushi Saito, Guangyu Song, Xiangru Tang, Bolun Wang, Johan S. Wind, Stanislaw Wozniak, Ruichong Zhang, Zhenyuan Zhang, Qihang Zhao, Peng Zhou, Qinghua Zhou, Jian Zhu, and Rui-Jie Zhu. 2023. RWKV: Reinventing RNNs for the transformer era. In *Proceedings of the Findings of ACL: EMNLP 2023*.
- [238] Baolin Peng, Chunyuan Li, Pengcheng He, Michel Galley, and Jianfeng Gao. 2023. Instruction tuning with gpt-4. arXiv:2304.03277. Retrieved from <https://arxiv.org/abs/2304.03277>. (2023).
- [239] Huwan Peng, Scott Davidson, Richard Shi, Shuaiwen Leon Song, and Michael Taylor. 2023. Chiplet Cloud: Building AI supercomputers for serving large generative language models. arXiv:2307.02666. Retrieved from <https://arxiv.org/abs/2307.02666>. (2023).
- [240] Reiner Pope, Sholto Douglas, Aakanksha Chowdhery, Jacob Devlin, James Bradbury, Jonathan Heek, Kefan Xiao, Shivani Agrawal, and Jeff Dean. 2023. Efficiently scaling transformer inference. *Proceedings of Machine Learning and Systems* (2023).
- [241] Ramya Prabhu, Ajay Nayak, Jayashree Mohan, Ramachandran Ramjee, and Ashish Panwar. 2025. vAttention: Dynamic memory management for serving LLMs without pagedattention. In *Proceedings of the ASPLOS 2025*. 1133–1150.
- [242] Ofir Press, Noah Smith, and Mike Lewis. 2021. Train short, test long: Attention with linear biases enables input length extrapolation. In *Proceedings of the ICLR 2021*.
- [243] Yifan Qiao, Shu Anzai, Shan Yu, Haoran Ma, Yang Wang, Miryung Kim, and Harry Xu. 2024. ConServe: Harvesting GPUs for low-latency and high-throughput large language model serving. arXiv:2410.01228. Retrieved from <https://arxiv.org/abs/2410.01228>. (2024).
- [244] Ruoyu Qin, Zheming Li, Weiran He, Mingxing Zhang, Yongwei Wu, Weimin Zheng, and Xinran Xu. 2024. Mooncake: A kvcache-centric disaggregated architecture for llm serving. arXiv:2407.00079. Retrieved from <https://arxiv.org/abs/2407.00079>. (2024).
- [245] Qualcomm. 2023. The future of AI is hybrid. Retrieved November 25, 2023 from <https://www.qualcomm.com/content/dam/qcomm-martech/dm-assets/documents/Whitepaper-The-future-of-AI-is-hybrid-Part-2-Qualcomm-is-uniquely-positioned-to-scale-hybrid-AI.pdf>. (2023).
- [246] Markus N. Rabe and Charles Staats. 2021. Self-attention does not need $O(n^2)$ memory. arXiv:2112.05682. Retrieved from <https://arxiv.org/abs/2112.05682>. (2021).
- [247] Samyam Rajbhandari, Conglong Li, Zhewei Yao, Minjia Zhang, Reza Yazdani Aminabadi, Ammar Ahmad Awan, Jeff Rasley, and Yuxiong He. 2022. Deepspeed-moe: Advancing mixture-of-experts inference and training to power next-generation ai scale. In *Proceedings of the ICML 2022*. 18332–18346.
- [248] Aditya Ramesh, Mikhail Pavlov, Gabriel Goh, Scott Gray, Chelsea Voss, Alec Radford, Mark Chen, and Ilya Sutskever. 2021. Zero-shot text-to-image generation. In *Proceedings of the ICML 2021*. 8821–8831.

- [249] Vijay Janapa Reddi, Christine Cheng, David Kanter, Peter Mattson, Guenther Schmuelling, Carole-Jean Wu, Brian Anderson, Maximilien Breughe, Mark Charlebois, William Chou, Ramesh Chukka, Cody Coleman, Sam Davis, Pan Deng, Greg Diamos, Jared Duke, Dave Fick, J. Scott Gardner, Itay Hubara, Sachin Idgunji, Thomas B. Jablin, Jeff Jiao, Tom St. John, Pankaj Kanwar, David Lee, Jeffery Liao, Anton Lokhmotov, Francisco Massa, Peng Meng, Paulius Micekevicius, Colin Osborne, Gennady Pekhimenko, Arun Tejusve Raghunath Rajan, Dilip Sequeira, Ashish Sirasao, Fei Sun, Hanlin Tang, Michael Thomson, Frank Wei, Ephrem Wu, Lingjie Xu, Koichi Yamada, Bing Yu, George Yuan, Aaron Zhong, Peizhao Zhang, and Yuchen Zhou. 2020. Mlperf inference benchmark. In *Proceedings of the ISCA 2020*. 446–459.
- [250] Stephen Roller, Sainbayar Sukhbaatar, Jason Weston, and Arthur Szlam. 2021. Hash layers for large sparse models. *Proceedings of NeurIPS (2021)*, 17555–17566.
- [251] Aurko Roy, Mohammad Saffar, Ashish Vaswani, and David Grangier. 2021. Efficient content-based sparse attention with routing transformers. *TACL (2021)*, 53–68.
- [252] Hasim Sak, Andrew W. Senior, and Françoise Beaufays. 2014. Long short-term memory recurrent neural network architectures for large scale acoustic modeling. In *Proceedings of the Interspeech*, Vol. 2014. 338–342.
- [253] Adrian Sampson, Tianqi Chen, and Jared Roesch. 2022. Apache TVM Unity: A vision for the ML software and hardware ecosystem. (2022).
- [254] Victor Sanh, Lysandre Debut, Julien Chaumond, and Thomas Wolf. 2019. DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter. arXiv:1910.01108. Retrieved from <https://arxiv.org/abs/1910.01108>. (2019).
- [255] Victor Sanh, Thomas Wolf, and Alexander Rush. 2020. Movement pruning: Adaptive sparsity by fine-tuning. *Proc. of NeurIPS (2020)*, 20378–20389.
- [256] Michael Santacrose, Zixin Wen, Yelong Shen, and Yuanzhi Li. 2023. What matters in the structured pruning of generative language models? arXiv:2302.03773. Retrieved from <https://arxiv.org/abs/2302.03773>. (2023).
- [257] Andrea Santilli, Silvio Severino, Emilian Postolache, Valentino Maiorca, Michele Mancusi, Riccardo Marin, and Emanuele Rodolà. 2023. Accelerating transformer inference for translation via parallel decoding. In *Proceedings of the ACL 2023*.
- [258] Cicero Nogueira dos Santos, James Lee-Thorp, Isaac Noble, Chung-Ching Chang, and David Uthus. 2023. Memory augmented language models through mixture of word experts. arXiv:2311.10768. Retrieved from <https://arxiv.org/abs/2311.10768>. (2023).
- [259] Tal Schuster, Adam Fisch, Tommi Jaakkola, and Regina Barzilay. 2021. Consistent accelerated inference via confident adaptive transformers. In *Proceedings of the EMNLP 2021*. 4962–4979.
- [260] Noam Shazeer. 2019. Fast transformer decoding: One write-head is all you need. arXiv:1911.02150. Retrieved from <https://arxiv.org/abs/1911.02150>. (2019).
- [261] Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. 2017. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. In *Proceedings of the ICLR 2017*.
- [262] Haihao Shen, Hanwen Chang, Bo Dong, Yu Luo, and Hengyu Meng. 2023. Efficient LLM inference on CPUs. arXiv:2311.00502. Retrieved from <https://arxiv.org/abs/2311.00502>. (2023).
- [263] Guangming Sheng, Chi Zhang, Zilingfeng Ye, Xibin Wu, Wang Zhang, Ru Zhang, Yanghua Peng, Haibin Lin, and Chuan Wu. 2024. Hybridflow: A flexible and efficient rlhf framework. arXiv:2409.19256. Retrieved from <https://arxiv.org/abs/2409.19256>. (2024).
- [264] Ying Sheng, Shiyi Cao, Dacheng Li, Coleman Hooper, Nicholas Lee, Shuo Yang, Christopher Chou, Banghua Zhu, Lianmin Zheng, Kurt Keutzer, Joseph E. Gonzalez, and Ion Stoica. 2023. S-LoRA: Serving thousands of concurrent LoRA adapters. arXiv:2311.03285. Retrieved from <https://arxiv.org/abs/2311.03285>. (2023).
- [265] Ying Sheng, Shiyi Cao, Dacheng Li, Banghua Zhu, Zhuohan Li, Danyang Zhuo, Joseph E. Gonzalez, and Ion Stoica. 2024. Fairness in serving large language models. In *Proceedings of the OSDI 2024*. 965–988.
- [266] Ying Sheng, Lianmin Zheng, Binhang Yuan, Zhuohan Li, Max Ryabinin, Beidi Chen, Percy Liang, Christopher Ré, Ion Stoica, and Ce Zhang. 2023. FlexGen: High-throughput generative inference of large language models with a single GPU. In *Proceedings of the ICML 2023*. 31094–31116.
- [267] Xing Shi and Kevin Knight. 2017. Speeding up neural machine translation decoding by shrinking run-time vocabulary. In *Proceedings of the ACL 2017*. 574–579.
- [268] Yining Shi, Zhi Yang, Jilong Xue, Lingxiao Ma, Yuqing Xia, Ziming Miao, Yuxiao Guo, Fan Yang, and Lidong Zhou. 2023. Welder: Scheduling deep learning memory access via tile-graph. In *Proceedings of the OSDI 2023*. 701–718.
- [269] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. 2019. Megatron-lm: Training multi-billion parameter language models using model parallelism. arXiv:1909.08053. Retrieved from <https://arxiv.org/abs/1909.08053>. (2019).
- [270] Yixin Song, Zeyu Mi, Haotong Xie, and Haibo Chen. 2023. PowerInfer: Fast large language model serving with a consumer-grade GPU. arXiv:2312.12456. Retrieved from <https://arxiv.org/abs/2312.12456>. (2023).

- [271] Benjamin Spector and Chris Re. 2023. Accelerating llm inference with staged speculative decoding. arXiv:2308.04623. Retrieved from <https://arxiv.org/abs/2308.04623>. (2023).
- [272] Mitchell Stern, Noam Shazeer, and Jakob Uszkoreit. 2018. Blockwise parallel decoding for deep autoregressive models. *Proc. of NeurIPS* (2018).
- [273] Jianlin Su, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, and Yunfeng Liu. 2021. Roformer: Enhanced transformer with rotary position embedding. arXiv:2104.09864. Retrieved from <https://arxiv.org/abs/2104.09864>. (2021).
- [274] Biao Sun, Ziming Huang, Hanyu Zhao, Wencong Xiao, Xinyi Zhang, Yong Li, and Wei Lin. 2024. Llumnix: Dynamic scheduling for large language model serving. In *Proceedings of the OSDI 2024*. 173–191.
- [275] Hanshi Sun, Zhuoming Chen, Xinyu Yang, Yuandong Tian, and Beidi Chen. 2024. TriForce: Lossless acceleration of long sequence generation with hierarchical speculative decoding. In *Proceedings of the COLM 2024*.
- [276] Mingjie Sun, Zhuang Liu, Anna Bair, and J. Zico Kolter. 2023. A simple and effective pruning approach for large language models. arXiv:2306.11695. Retrieved from <https://arxiv.org/abs/2306.11695>. (2023).
- [277] Siqi Sun, Yu Cheng, Zhe Gan, and Jingjing Liu. 2019. Patient knowledge distillation for BERT model compression. In *Proceedings of the EMNLP-IJCNLP 2019*. 4323–4332.
- [278] Tianxiang Sun, Xiangyang Liu, Wei Zhu, Zhichao Geng, Lingling Wu, Yilong He, Yuan Ni, Guotong Xie, Xuanjing Huang, and Xipeng Qiu. 2022. A simple hash-based early exiting approach for language understanding and generation. arXiv:2203.01670. Retrieved from <https://arxiv.org/abs/2203.01670>. (2022).
- [279] Yutao Sun, Li Dong, Shaohan Huang, Shuming Ma, Yuqing Xia, Jilong Xue, Jianyong Wang, and Furu Wei. 2023. Retentive network: A successor to transformer for large language models. arXiv:2307.08621. Retrieved from <https://arxiv.org/abs/2307.08621>. (2023).
- [280] Ziteng Sun, Ananda Theertha Suresh, Jae Hun Ro, Ahmad Beirami, Himanshu Jain, Felix Yu, Michael Riley, and Sanjiv Kumar. 2023. Spectr: Fast speculative decoding via optimal transport. In *Proceedings of the Workshop on Efficient Systems for Foundation Models@ ICML2023*.
- [281] Zhenheng Tang, Yuxin Wang, Xin He, Longteng Zhang, Xinglin Pan, Qiang Wang, Rongfei Zeng, Kaiyong Zhao, Shaohuai Shi, Bingsheng He, and Xiaowen Chu. 2023. FusionAI: Decentralized training and deploying LLMs with massive consumer-level GPUs. arXiv:2309.01172. Retrieved from <https://arxiv.org/abs/2309.01172>. (2023).
- [282] Rohan Taori, Ishaan Gulrajani, Tianyi Zhang, Yann Dubois, Xuechen Li, Carlos Guestrin, Percy Liang, and Tatsunori B. Hashimoto. 2023. Stanford alpaca: An instruction-following llama model. (2023).
- [283] Yi Tay, Dara Bahri, Liu Yang, Donald Metzler, and Da-Cheng Juan. 2020. Sparse sinkhorn attention. In *Proceedings of the ICML 2020*.
- [284] Yi Tay, Mostafa Dehghani, Dara Bahri, and Donald Metzler. 2023. Efficient transformers: A survey. *ACM Computing Surveys* (2023), 109:1–109:28.
- [285] MLC team. 2023. MLC-LLM. (2023). Retrieved November 25, 2023 from <https://github.com/mlc-ai/mlc-llm> Commit: 3358029.
- [286] Surat Teerapittayanon, Bradley McDanel, and Hsiang-Tsung Kung. 2016. Branchynet: Fast inference via early exiting from deep neural networks. In *Proceedings of the ICPR 2016*. 2464–2469.
- [287] Philippe Tillet, Hsiang-Tsung Kung, and David Cox. 2019. Triton: An intermediate language and compiler for tiled neural network computations. In *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages*. 10–19.
- [288] Ilya O. Tolstikhin, Neil Houlsby, Alexander Kolesnikov, Lucas Beyer, Xiaohua Zhai, Thomas Unterthiner, Jessica Yung, Andreas Steiner, Daniel Keysers, Jakob Uszkoreit, Mario Lucic, and Alexey Dosovitskiy. 2021. Mlp-mixer: An all-mlp architecture for vision. *Proceedings of NeurIPS* (2021), 24261–24272.
- [289] Alexander Tornede, Difan Deng, Theresa Eimer, Joseph Giovanelli, Aditya Mohan, Tim Rühkopf, Sarah Segel, Daphne Theodorakopoulos, Tanja Tornede, Henning Wachsmuth, and Marius Lindauer. 2023. AutoML in the age of large language models: Current challenges, future opportunities and risks. arXiv:2306.08107. Retrieved from <https://arxiv.org/abs/2306.08107>. (2023).
- [290] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajwal Bhargava, Shruti Bhosale, Dan Bikel, Lukas Blecher, Cristian Canton Ferrer, Moya Chen, Guillem Cucurull, David Esiobu, Jude Fernandes, Jeremy Fu, Wenyin Fu, Brian Fuller, Cynthia Gao, Vedanuj Goswami, Naman Goyal, Anthony Hartshorn, Saghar Hosseini, Rui Hou, Hakan Inan, Marcin Kardas, Viktor Kerkez, Madian Khabsa, Isabel Kloumann, Artem Korenev, Punit Singh Koura, Marie-Anne Lachaux, Thibaut Lavril, Jenya Lee, Diana Liskovich, Yinghai Lu, Yuning Mao, Xavier Martinet, Todor Mihaylov, Pushkar Mishra, Igor Molybog, Yixin Nie, Andrew Poulton, Jeremy Reizenstein, Rashi Rungta, Kalyan Saladi, Alan Schelten, Ruan Silva, Eric Michael Smith, Ranjan Subramanian, Xiaoqing Ellen Tan, Binh Tang, Ross Taylor, Adina Williams, Jian Xiang Kuan, Puxin Xu, Zheng Yan, Iliyan Zarov, Yuchen Zhang, Angela Fan, Melanie Kambadur, Sharan Narang, Aurelien Rodriguez, Robert Stojnic, Sergey Edunov, and Thomas Scialom. 2023. Llama 2: Open foundation and fine-tuned chat models. arXiv:2307.09288. Retrieved from <https://arxiv.org/abs/2307.09288>. (2023).

- [291] Marcos Treviso, Ji-Ung Lee, Tianchu Ji, Betty van Aken, Qingqing Cao, Manuel R. Ciosici, Michael Hassid, Kenneth Heafield, Sara Hooker, Colin Raffel, Pedro H. Martins, André F. T. Martins, Jessica Zosa Forde, Peter Milder, Edwin Simpson, Noam Slonim, Jesse Dodge, Emma Strubell, Niranjan Balasubramanian, Leon Derczynski, Iryna Gurevych, and Roy Schwartz. 2023. Efficient methods for natural language processing: A survey. *TACL* (2023), 826–860.
- [292] Francisco Massa Grigory Sizov Tri Dao, Daniel Haziza. 2023. Flash-Decoding for long-context inference. (2023). Retrieved from <https://pytorch.org/blog/flash-decoding/>
- [293] Colin Unger, Zhihao Jia, Wei Wu, Sina Lin, Mandeep Baines, Carlos Efrain Quintero Narvaez, Vinay Ramakrishnaiah, Nirmal Prajapati, Pat McCormick, Jamaludin Mohd-Yusof, Xi Luo, Dheevatsa Mudigere, Jongsoo Park, Misha Smelyanskiy, and Alex Aiken. 2022. Unity: Accelerating DNN training through joint optimization of algebraic transformations and parallelization. In *Proceedings of the OSDI 2022*. 267–284.
- [294] Tim Valicenti, Justice Vidal, and Ritik Patnaik. 2023. Mini-GPTs: Efficient large language models through contextual pruning. arXiv:2312.12682. Retrieved from <https://arxiv.org/abs/2312.12682>. (2023).
- [295] Robert A. Van De Geijn and Jerrell Watts. 1997. SUMMA: Scalable universal matrix multiplication algorithm. *Concurrency: Practice and Experience* (1997), 255–274.
- [296] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. In *Proceedings of the NeurIPS 2017*.
- [297] Jikai Wang, Yi Su, Juntao Li, Qingrong Xia, Zi Ye, Xinyu Duan, Zhefeng Wang, and Min Zhang. 2025. Opt-tree: Speculative decoding with adaptive draft tree structure. *TACL* (2025), 188–199.
- [298] Sinong Wang, Belinda Z. Li, Madian Khabza, Han Fang, and Hao Ma. 2020. Linformer: Self-attention with linear complexity. arXiv:2006.04768. Retrieved from <https://arxiv.org/abs/2006.04768>. (2020).
- [299] Wenhui Wang, Furu Wei, Li Dong, Hangbo Bao, Nan Yang, and Ming Zhou. 2020. Minilm: Deep self-attention distillation for task-agnostic compression of pre-trained transformers. *Proc. of NeurIPS* (2020), 5776–5788.
- [300] Xiaohui Wang, Ying Xiong, Yang Wei, Mingxuan Wang, and Lei Li. 2020. LightSeq: A high performance inference library for transformers. arXiv:2010.13887. Retrieved from <https://arxiv.org/abs/2010.13887>. (2020).
- [301] Yiding Wang, Kai Chen, Haisheng Tan, and Kun Guo. 2023. Tabi: An efficient multi-level inference system for large language models. In *Proceedings of the EuroSys 2023*. 233–248.
- [302] Yuxin Wang, Yuhan Chen, Zeyu Li, Xueze Kang, Zhenheng Tang, Xin He, Rui Guo, Xin Wang, Qiang Wang, Amelie Chi Zhou, Yuchu Fang, Yeju Zhou, Yang Zheng, and Xiaowen Chu. 2024. BurstGPT: A real-world workload dataset to optimize LLM serving systems. arXiv:2401.17644. Retrieved from <https://arxiv.org/abs/2401.17644>. (2024).
- [303] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V. Le, Denny Zhou, et al. 2022. Chain-of-thought prompting elicits reasoning in large language models. *Proceedings of NeurIPS* (2022), 24824–24837.
- [304] Qizhen Weng, Wencong Xiao, Yinghao Yu, Wei Wang, Cheng Wang, Jian He, Yong Li, Liping Zhang, Wei Lin, and Yu Ding. 2022. MLaaS in the wild: Workload analysis and scheduling in Large-Scale heterogeneous GPU clusters. In *Proceedings of the NSDI 2022*. 945–960.
- [305] BigScience Workshop, Teven Le Scao, Angela Fan, Christopher Akiki, Ellie Pavlick, Suzana Ilić, Daniel Hesslow, Roman Castagné, Alexandra Sasha Luccioni, François Yvon. 2022. Bloom: A 176b-parameter open-access multilingual language model. arXiv:2211.05100. Retrieved from <https://arxiv.org/abs/2211.05100>. (2022).
- [306] Bingyang Wu, Shengyu Liu, Yinmin Zhong, Peng Sun, Xuanzhe Liu, and Xin Jin. 2024. Loongserve: Efficiently serving long-context large language models with elastic sequence parallelism. In *Proceedings of the SOSP 2024*. 640–654.
- [307] Bingyang Wu, Yinmin Zhong, Zili Zhang, Gang Huang, Xuanzhe Liu, and Xin Jin. 2023. Fast distributed inference serving for large language models. arXiv:2305.05920. Retrieved from <https://arxiv.org/abs/2305.05920>. (2023).
- [308] Bingyang Wu, Ruidong Zhu, Zili Zhang, Peng Sun, Xuanzhe Liu, and Xin Jin. 2024. dLoRA: Dynamically orchestrating requests and adapters for LoRA LLM serving. In *Proceedings of the OSDI 2024*. 911–927.
- [309] Kaixin Wu, Bojie Hu, and Qi Ju. 2021. TenTrans high-performance inference toolkit for WMT2021 efficiency task. In *Proceedings of the 6th Conference on Machine Translation*. 795–798.
- [310] Kaixin Wu, Yue Zhang, Bojie Hu, and Tong Zhang. 2022. Speeding up transformer decoding via an attention refinement network. In *Proceedings of the COLING 2022*. 5109–5118.
- [311] Mengdi Wu, Xinhao Cheng, Shengyu Liu, Chunan Shi, Jianan Ji, Kit Ao, Praveen Velliengiri, Xupeng Miao, Oded Padon, and Zhihao Jia. 2024. Mirage: A multi-level superoptimizer for tensor programs. arXiv:2405.05751. Retrieved from <https://arxiv.org/abs/2405.05751>. (2024).
- [312] Peng Wu. 2023. PyTorch 2.0: The journey to bringing compiler technologies to the core of pytorch (keynote). In *Proceedings of the 21st ACM/IEEE International Symposium on Code Generation and Optimization*. 1–1.
- [313] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Shaokun Zhang, Erkang Zhu, Beibin Li, Li Jiang, Xiaoyun Zhang, and Chi Wang. 2023. Autogen: Enabling next-gen llm applications via multi-agent conversation framework. arXiv:2308.08155. Retrieved from <https://arxiv.org/abs/2308.08155>. (2023).
- [314] Xiaoxia Wu, Cheng Li, Reza Yazdani Aminabadi, Zhewei Yao, and Yuxiong He. 2023. Understanding INT4 quantization for transformer models: Latency speedup, composability, and failure cases. arXiv:2301.12017. Retrieved from <https://arxiv.org/abs/2301.12017>. (2023).

- [315] Haojun Xia, Zhen Zheng, Yuchao Li, Donglin Zhuang, Zhongzhu Zhou, Xiafei Qiu, Yong Li, Wei Lin, and Shuaiwen Leon Song. 2023. Flash-LLM: Enabling cost-effective and highly-efficient large generative model inference with unstructured sparsity. arXiv:2309.10285. Retrieved from <https://arxiv.org/abs/2309.10285>. (2023).
- [316] Guangxuan Xiao, Ji Lin, Mickael Seznec, Julien Demouth, and Song Han. 2022. Smoothquant: Accurate and efficient post-training quantization for large language models. arXiv:2211.10438. Retrieved from <https://arxiv.org/abs/2211.10438>. (2022).
- [317] Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. 2023. Efficient streaming language models with attention sinks. arXiv:2309.17453. Retrieved from <https://arxiv.org/abs/2309.17453>. (2023).
- [318] Tong Xiao, Yinqiao Li, Jingbo Zhu, Zhengtao Yu, and Tongran Liu. 2019. Sharing attention weights for fast transformer. In *Proceedings of the IJCAI 2019*. 5292–5298.
- [319] Yisheng Xiao, Lijun Wu, Junliang Guo, Juntao Li, Min Zhang, Tao Qin, and Tie-yan Liu. 2023. A survey on non-autoregressive generation for neural machine translation and beyond. *IEEE TPAMI* (2023).
- [320] Zhiqiang Xie, Hao Kang, Ying Sheng, Tushar Krishna, Kayvon Fatahalian, and Christos Kozyrakis. 2024. AI metropolis: Scaling large language model-based multi-agent simulation with out-of-order execution. arXiv:2411.03519. Retrieved from <https://arxiv.org/abs/2411.03519>. (2024).
- [321] Ji Xin, Raphael Tang, Jaejun Lee, Yaoliang Yu, and Jimmy Lin. 2020. DeeBERT: Dynamic early exiting for accelerating BERT inference. In *Proceedings of the ACL 2020*. 2246–2251.
- [322] Can Xu, Qingfeng Sun, Kai Zheng, Xiubo Geng, Pu Zhao, Jiazhan Feng, Chongyang Tao, and Daxin Jiang. 2023. Wizardlm: Empowering large language models to follow complex instructions. arXiv:2304.12244. Retrieved from <https://arxiv.org/abs/2304.12244>. (2023).
- [323] Daliang Xu, Wangsong Yin, Xin Jin, Ying Zhang, Shiyun Wei, Mengwei Xu, and Xuanzhe Liu. 2023. LLMcad: Fast and scalable on-device large language model inference. arXiv:2309.04255. Retrieved from <https://arxiv.org/abs/2309.04255>. (2023).
- [324] Peng Xu, Wei Ping, Xianchao Wu, Lawrence McAfee, Chen Zhu, Zihan Liu, Sandeep Subramanian, Evelina Bakhurina, Mohammad Shoeybi, and Bryan Catanzaro. 2023. Retrieval meets Long Context Large Language Models. arXiv:2310.03025. Retrieved from <https://arxiv.org/abs/2310.03025>. (2023).
- [325] Zhaozhuo Xu, Zirui Liu, Beidi Chen, Yuxin Tang, Jue Wang, Kaixiong Zhou, Xia Hu, and Anshumali Shrivastava. 2023. Compress, then prompt: Improving accuracy-efficiency trade-off of LLM inference with transferable prompt. arXiv:2305.11186. Retrieved from <https://arxiv.org/abs/2305.11186>. (2023).
- [326] Aiyuan Yang, Bin Xiao, Bingning Wang, Borong Zhang, Chao Yin, Chenxu Lv, Da Pan, Dian Wang, Dong Yan, Fan Yang, Ce Bian, Fei Deng, Feng Wang, Feng Liu, Guangwei Ai, Guosheng Dong, Haizhou Zhao, Hang Xu, Haoze Sun, Hongda Zhang, Hui Liu, Jiaming Ji, Jian Xie, JunTao Dai, Kun Fang, Lei Su, Liang Song, Lifeng Liu, Liyun Ru, Luyao Ma, Mang Wang, Mickel Liu, MingAn Lin, Nuolan Nie, Peidong Guo, Ruiyang Sun, Tao Zhang, Tianpeng Li, Tianyu Li, Wei Cheng, Weipeng Chen, Xiangrong Zeng, Xiaochuan Wang, Xiaoxi Chen, Xin Men, Xin Yu, Xuehai Pan, Yanjun Shen, Yiding Wang, Yiyu Li, Youxin Jiang, Yuchen Gao, Yupeng Zhang, Zenan Zhou, and Zhiying Wu. 2023. Baichuan 2: Open large-scale language models. arXiv:2309.10305. Retrieved from <https://arxiv.org/abs/2309.10305>. (2023).
- [327] Amy Yang, Jingyi Yang, Aya Ibrahim, Xinfeng Xie, Bangsheng Tang, Grigory Sizov, Jeremy Reizenstein, Jongsoo Park, and Jianyu Huang. 2024. Context parallelism for scalable million-token inference. arXiv:2411.01783. Retrieved from <https://arxiv.org/abs/2411.01783>. (2024).
- [328] Nan Yang, Tao Ge, Liang Wang, Binxing Jiao, Daxin Jiang, Linjun Yang, Rangan Majumder, and Furu Wei. 2023. Inference with reference: Lossless acceleration of large language models. arXiv:2304.04487. Retrieved from <https://arxiv.org/abs/2304.04487>. (2023).
- [329] Penghui Yang, Cunxiao Du, Fengzhuo Zhang, Haonan Wang, Tianyu Pang, Chao Du, and Bo An. 2025. LongSpec: Long-context speculative decoding with efficient drafting and verification. arXiv:2502.17421. Retrieved from <https://arxiv.org/abs/2502.17421>. (2025).
- [330] Seongjun Yang, Gibbeum Lee, Jaewoong Cho, Dimitris Papailiopoulos, and Kangwook Lee. 2023. Predictive pipelined decoding: A compute-latency trade-off for exact LLM decoding. arXiv:2307.05908. Retrieved from <https://arxiv.org/abs/2307.05908>. (2023).
- [331] Zhewei Yao, Cheng Li, Xiaoxia Wu, Stephen Youn, and Yuxiong He. 2023. A comprehensive study on post-training quantization for large language models. arXiv:2303.08302. Retrieved from <https://arxiv.org/abs/2303.08302>. (2023).
- [332] Zhewei Yao, Reza Yazdani Aminabadi, Minjia Zhang, Xiaoxia Wu, Conglong Li, and Yuxiong He. 2022. Zeroquant: Efficient and affordable post-training quantization for large-scale transformers. *Proceedings of NeurIPS* (2022), 27168–27183.
- [333] Deming Ye, Yankai Lin, Yufei Huang, and Maosong Sun. 2021. TR-BERT: Dynamic token reduction for accelerating bert inference. In *Proceedings of the NAACL 2021*. 5798–5809.
- [334] Zihao Ye, Lequn Chen, Ruihang Lai, Wuwei Lin, Yineng Zhang, Stephanie Wang, Tianqi Chen, Baris Kasikci, Vinod Grover, Arvind Krishnamurthy, and Luis Ceze. 2025. Flashinfer: Efficient and customizable attention engine for llm inference serving. arXiv:2501.01005. Retrieved from <https://arxiv.org/abs/2501.01005>. (2025).

- [335] Zihao Ye, Ruihang Lai, Junru Shao, Tianqi Chen, and Luis Ceze. 2023. SparseTIR: Composable abstractions for sparse compilation in deep learning. In *Proceedings of the ASPLOS 2023*. 660–678.
- [336] Anil Yemme and Shayan Srinivasa Garani. 2023. A scalable GPT-2 inference hardware architecture on FPGA. In *Proceedings of the IJCNN 2023*. 1–8.
- [337] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. 2022. Orca: A distributed serving system for transformer-based generative models. In *Proceedings of the OSDI 2022*. 521–538.
- [338] Lingfan Yu, Jinkun Lin, and Jinyang Li. 2023. Stateful large language model serving with pensieve. arXiv:2312.05516. Retrieved from <https://arxiv.org/abs/2312.05516>. (2023).
- [339] Weihao Yu, Mi Luo, Pan Zhou, Chenyang Si, Yichen Zhou, Xinchao Wang, Jiashi Feng, and Shuicheng Yan. 2022. Metaformer is actually what you need for vision. In *Proceedings of the CVPR 2022*. 10819–10829.
- [340] Zhihang Yuan, Lin Niu, Jiawei Liu, Wenyu Liu, Xinggang Wang, Yuzhang Shang, Guangyu Sun, Qiang Wu, Jiaxiang Wu, and Bingzhe Wu. 2023. RPTQ: Reorder-based Post-training quantization for large language models. arXiv:2304.01089. Retrieved from <https://arxiv.org/abs/2304.01089>. (2023).
- [341] Murong Yue, Jie Zhao, Min Zhang, Du Liang, and Ziyu Yao. 2024. Large language model cascades with mixture of thoughts representations for cost-efficient reasoning. In *Proceedings of the ICLR 2024*.
- [342] Manzil Zaheer, Guru Guruganesh, Kumar Avinava Dubey, Joshua Ainslie, Chris Alberti, Santiago Ontanon, Philip Pham, Anirudh Ravula, Qifan Wang, Li Yang, and Amr Ahmed. 2020. Big bird: Transformers for longer sequences. *Proceedings of NeurIPS (2020)*, 17283–17297.
- [343] Aohan Zeng, Xiao Liu, Zhengxiao Du, Zihan Wang, Hanyu Lai, Ming Ding, Zhuoyi Yang, Yifan Xu, Wendi Zheng, Xiao Xia, Weng Lam Tam, Zixuan Ma, Yufei Xue, Jidong Zhai, Wenguang Chen, Peng Zhang, Yuxiao Dong, and Jie Tang. 2022. Glm-130b: An open bilingual pre-trained model. arXiv:2210.02414. Retrieved from <https://arxiv.org/abs/2210.02414>. (2022).
- [344] Dewen Zeng, Nan Du, Tao Wang, Yuanzhong Xu, Tao Lei, Zhifeng Chen, and Claire Cui. 2023. Learning to skip for language modeling. arXiv:2311.15436. Retrieved from <https://arxiv.org/abs/2311.15436>. (2023).
- [345] Shuangfei Zhai, Walter Talbott, Nitish Srivastava, Chen Huang, Hanlin Goh, Ruixiang Zhang, and Josh Susskind. 2021. An attention free transformer. arXiv:2105.14103. Retrieved from <https://arxiv.org/abs/2105.14103>. (2021).
- [346] Yujia Zhai, Chengquan Jiang, Leyuan Wang, Xiaoying Jia, Shang Zhang, Zizhong Chen, Xin Liu, and Yibo Zhu. 2023. Bytetransformer: A high-performance transformer boosted for variable-length inputs. In *Proceedings of the IPDPS 2023*. 344–355.
- [347] Jiaao Zhan, Qian Chen, Boxing Chen, Wen Wang, Yu Bai, and Yang Gao. 2023. DePA: Improving non-autoregressive translation with dependency-aware decoder. In *Proceedings of the IWSLT 2023*. 478–490.
- [348] Chengliang Zhang, Minchen Yu, Wei Wang, and Feng Yan. 2019. MARK: Exploiting cloud services for cost-effective, SLO-aware machine learning inference serving. In *Proceedings of the USENIX ATC 2019*. 1049–1062.
- [349] Hailin Zhang, Xiaodong Ji, Yilin Chen, Fangcheng Fu, Xupeng Miao, Xiaonan Nie, Weipeng Chen, and Bin Cui. 2024. Pqcache: Product quantization-based kvcache for long context llm inference. arXiv:2407.12820. Retrieved from <https://arxiv.org/abs/2407.12820>. (2024).
- [350] Jun Zhang, Jue Wang, Huan Li, Lidan Shou, Ke Chen, Gang Chen, and Sharad Mehrotra. 2024. Draft& Verify: Lossless large language model acceleration via self-speculative decoding. In *Proceedings of the ACL 2024*. 11263–11282.
- [351] Longteng Zhang, Xiang Liu, Zeyu Li, Xinglin Pan, Peijie Dong, Ruibo Fan, Rui Guo, Xin Wang, Qiong Luo, Shaohuai Shi, and Xiaowen Chu. 2023. Dissecting the runtime performance of the training, fine-tuning, and inference of large language models. arXiv:2311.03687. Retrieved from <https://arxiv.org/abs/2311.03687>. (2023).
- [352] Mengke Zhang, Tianxing He, Tianle Wang, Fatemehsadat Miresghallah, Binyi Chen, Hao Wang, and Yulia Tsvetkov. 2023. LatticeGen: A cooperative framework which hides generated text in a lattice for privacy-aware generation on cloud. arXiv:2309.17157. Retrieved from <https://arxiv.org/abs/2309.17157>. (2023).
- [353] Qingru Zhang, Dhananjay Ram, Cole Hawkins, Sheng Zha, and Tuo Zhao. 2023. Efficient long-range transformers: You need to attend more, but not necessarily at every layer. In *Proceedings of the EMNLP Findings*. 2775–2786.
- [354] Susan Zhang, Stephen Roller, Naman Goyal, Mikel Artetxe, Moya Chen, Shuohui Chen, Christopher Dewan, Mona Diab, Xian Li, Xi Victoria Lin, Todor Mihaylov, Myle Ott, Sam Shleifer, Kurt Shuster, Daniel Simig, Punit Singh Koura, Anjali Sridhar, Tianlu Wang, and Luke Zettlemoyer. 2022. Opt: Open pre-trained transformer language models. arXiv:2205.01068. Retrieved from <https://arxiv.org/abs/2205.01068>. (2022).
- [355] Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, Zhangyang Wang, and Beidi Chen. 2023. H₂O: Heavy-hitter oracle for efficient generative inference of large language models. arXiv:2306.14048. Retrieved from <https://arxiv.org/abs/2306.14048>. (2023).
- [356] Zhihao Zhang, Alan Zhu, Lijie Yang, Yihua Xu, Lanting Li, Phitchaya Mangpo Phothilimthana, and Zhihao Jia. 2024. Accelerating iterative retrieval-augmented language model serving with speculation. In *Proceedings of the ICML 2024*.

- [357] Chenggang Zhao, Shangyan Zhou, Liyue Zhang, Chengqi Deng, Zhean Xu, Yuxuan Liu, Kuai Yu, Jiashi Li, and Liang Zhao. 2025. DeepEP: An efficient expert-parallel communication library. Retrieved from <https://github.com/deepseek-ai/DeepEP>. (2025).
- [358] Weilin Zhao, Yuxiang Huang, Xu Han, Wang Xu, Chaojun Xiao, Xinrong Zhang, Yewei Fang, Kaihuo Zhang, Zhiyuan Liu, and Maosong Sun. 2024. Ouroboros: Generating longer drafts phrase by phrase for faster speculative decoding. In *Proceedings of the EMNLP 2024*. 13378–13393.
- [359] Yilong Zhao, Chien-Yu Lin, Kan Zhu, Zihao Ye, Lequn Chen, Size Zheng, Luis Ceze, Arvind Krishnamurthy, Tianqi Chen, and Baris Kasikci. 2024. Atom: Low-bit quantization for efficient and accurate llm serving. *Proceedings of Machine Learning and Systems* 6 (2024), 196–209.
- [360] Lianmin Zheng, Zhuohan Li, Hao Zhang, Yonghao Zhuang, Zhifeng Chen, Yanping Huang, Yida Wang, Yuanzhong Xu, Danyang Zhuo, Eric P. Xing, Joseph E. Gonzalez, and Ion Stoica. 2022. Alpa: Automating inter-and Intra-Operator parallelism for distributed deep learning. In *Proceedings of the OSDI 2022*. 559–578.
- [361] Liyan Zheng, Haojie Wang, Jidong Zhai, Muyan Hu, Zixuan Ma, Tuowei Wang, Shuhong Huang, Xupeng Miao, Shizhi Tang, Kezhao Huang, and Zhihao Jia. 2023. EINNNet: Optimizing tensor programs with derivation-based transformations. In *Proceedings of the OSDI 2023*. 739–755.
- [362] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Jeff Huang, Chuyue Sun, Cody_Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. 2023. Efficiently programming large language models using SGLang. (2023).
- [363] Ningxin Zheng, Huiqiang Jiang, Quanlu Zhang, Zhenhua Han, Lingxiao Ma, Yuqing Yang, Fan Yang, Chengruidong Zhang, Lili Qiu, Mao Yang, and Lidong Zhou. 2023. PIT: Optimization of dynamic sparse deep learning models via permutation invariant transformation. In *Proceedings of the SOSP 2023*. 331–347.
- [364] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. 2024. DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving. In *Proceedings of the OSDI 2024*. 193–210.
- [365] Yinmin Zhong, Zili Zhang, Bingyang Wu, Shengyu Liu, Yukun Chen, Changyi Wan, Hanpeng Hu, Lei Xia, Ranchen Ming, Yibo Zhu, and Xin Jin. 2024. Rlhfuse: Efficient rlhf training for large language models with inter-and intra-stage fusion. arXiv:2409.13221. Retrieved from <https://arxiv.org/abs/2409.13221>. (2024).
- [366] Haoyi Zhou, Shanghang Zhang, Jieqi Peng, Shuai Zhang, Jianxin Li, Hui Xiong, and Wancai Zhang. 2021. Informer: Beyond efficient transformer for long sequence time-series forecasting. In *Proceedings of the AAAI 2021*. 11106–11115.
- [367] Minxuan Zhou, Weihong Xu, Jaeyoung Kang, and Tajana Rosing. 2022. Transpim: A memory-based acceleration via software-hardware co-design for transformer. In *Proceedings of the HPCA 2022*. 1071–1085.
- [368] Wangchunshu Zhou, Canwen Xu, Tao Ge, Julian McAuley, Ke Xu, and Furu Wei. 2020. Bert loses patience: Fast and robust inference with early exit. *Proceedings of NeurIPS* (2020), 18330–18341.
- [369] Yanqi Zhou, Tao Lei, Hanxiao Liu, Nan Du, Yanping Huang, Vincent Zhao, Andrew M. Dai, Quoc V. Le, James Laudon, and Zhifeng Chen. 2022. Mixture-of-experts with expert choice routing. *Proceedings of NeurIPS* (2022), 7103–7114.
- [370] Yongchao Zhou, Kaifeng Lyu, Ankit Singh Rawat, Aditya Krishna Menon, Afshin Rostamizadeh, Sanjiv Kumar, Jean-François Kary, and Rishabh Agarwal. 2023. DistillSpec: Improving speculative decoding via knowledge distillation. arXiv:2310.08461. Retrieved from <https://arxiv.org/abs/2310.08461>. (2023).
- [371] Zhe Zhou, Xuechao Wei, Jiejing Zhang, and Guangyu Sun. 2022. PetS: A unified framework for parameter-efficient transformers serving. In *Proceedings of the USENIX ATC 2022*. 489–504.
- [372] Banghua Zhu, Ying Sheng, Lianmin Zheng, Clark Barrett, Michael I. Jordan, and Jiantao Jiao. 2023. On optimal caching and model multiplexing for large model inference. arXiv:2306.02003. Retrieved from <https://arxiv.org/abs/2306.02003>. (2023).
- [373] Deyao Zhu, Jun Chen, Xiaoqian Shen, Xiang Li, and Mohamed Elhoseiny. 2023. Minigpt-4: Enhancing vision-language understanding with advanced large language models. arXiv:2304.10592. Retrieved from <https://arxiv.org/abs/2304.10592>. (2023).
- [374] Xunyu Zhu, Jian Li, Yong Liu, Can Ma, and Weiping Wang. 2023. A survey on model compression for large language models. arXiv:2308.07633. Retrieved from <https://arxiv.org/abs/2308.07633>. (2023).
- [375] Yoshua X. Z Zhang, Yann M. Haxo, and Ying X. Mat. 2023. Falcon LLM: A new frontier in natural language processing. *AC Investment Research Journal* (2023).

Received 23 December 2023; revised 19 March 2025; accepted 16 July 2025